



中南大學
CENTRAL SOUTH UNIVERSITY

嵌入式课程实验报告

学生姓名：	樊博文
指导教师：	张航
学 号：	8207230322
学 院：	自动化学院
专业班级：	电子信息工程 2303

本科生院制

2026 年 5 月

目录

改进实验说明表	3
LED 彩灯实验	4
1、实验目的	4
2、实验环境	4
3、实验原理	4
4、实验步骤	5
5、实验代码	8
6、实验现象	12
原实验	12
改进实验	12
7、心得体会	14
蜂鸣器实验	15
1、实验目的	15
2、实验环境	15
3、实验原理	15
4、实验步骤	16
5、实验代码	16
6、实验现象	20
原实验	20
改进实验	20
7、心得体会	21
串口输出实验	23
1、实验目的	23
2、实验环境	23
3、实验原理	23
4、实验步骤	25
5、实验代码	26
6、实验现象	27
原实验	27
改进实验	28
7、心得体会	29
串口输入实验	30
1、实验目的	30
2、实验环境	30
3、实验原理	30
4、实验步骤	30
5、实验代码	31
6、实验现象	33
原实验	33
改进实验	34
7、心得体会	36

复位实验	37
1、实验环境	37
2、实验目的	37
3、实验原理	37
4、实验步骤	38
5、实验代码	39
6、实验现象	42
原实验	42
改进实验	44
7、心得体会	45
按键中断实验	47
1、实验环境	47
2、实验目的	47
3、实验原理	47
4、实验步骤	48
5、实验代码	48
6、实验现象	53
原实验	53
改进实验	55
7、心得体会	56
按键轮询实验	57
1、实验目的	57
2、实验环境	57
3、实验原理	57
4、实验步骤	58
5、实验代码	59
6、实验现象	65
原实验	65
改进实验	65
7、心得体会	68
A/D 实验	69
1、实验环境	69
2、实验目的	69
3、实验原理	69
4、实验内容	69
5、实验步骤	70
6、实验代码	71
7、实验现象	77
原实验	77
改进实验	78
8、心得体会	79

改进实验说明表

实验名称	原实验功能	改进内容	改进后实现效果
LED 彩灯实验	原实验通过 GPIO 控制 LED 灯闪烁, 主要实现单一或简单的亮灭控制。	将 LED 控制封装为 r_led()、g_led()、b_led() 函数, 并通过延时函数控制红、绿、蓝三色 LED 依次点亮。	RGB LED 能够按照红、绿、蓝的顺序循环闪烁, 程序结构更加清晰, 也便于后续修改闪烁频率和控制方式。
蜂鸣器实验	原实验通过 PWM 控制蜂鸣器发出固定频率的声音。	增加音符频率数组和持续时间数组, 通过 PWM 输出不同频率的方波, 使蜂鸣器按照设定旋律发声。	蜂鸣器不再发出单一声音, 而是能够播放《小星星》前两句旋律, 增强了实验的趣味性和功能性。
串口输出实验	原实验通过串口输出固定测试字符串。	将输出内容修改为包含姓名、班级、学号的信息, 并在循环中定时重复发送。	串口调试助手中能够周期性显示自定义信息, 验证了串口初始化和字符串发送功能。
串口输入实验	原实验每接收到一个字符就立即回显, 主要实现单字节接收。	增加缓冲区, 将接收到的数据先保存起来, 检测到回车或换行后再整体回传。	能够实现英文、数字以及中文字符串的整行接收和回显, 避免中文字符按单字节显示时可能出现乱码或显示不完整的问题。
复位实验	原实验通过软件方式触发系统复位, 并在串口中显示复位提示信息。	在触发复位前增加 LED 状态指示, 使复位前的程序执行过程更加直观。	程序运行后先点亮红色 LED 并输出提示信息, 随后触发系统复位, 串口显示 U-Boot 启动信息, 说明复位成功。
按键中断实验	原实验通过按键中断触发程序响应, 并在串口输出按键信息。	将按键中断与 RGB LED 控制结合起来, 按键触发后由主程序检测状态变量并控制红灯亮灭。	按下按键后, 串口输出提示信息, 同时红色 LED 短暂亮起后熄灭, 体现了中断触发和主程序处理相结合的过程。
按键轮询实验	原实验通过轮询方式检测按键状态, 并控制 LED 状态变化。	增加 led_mode 状态变量, 使每次按键后切换不同颜色的 LED, 并在串口输出当前 LED 状态。	每次按下按键时, LED 按红灯、绿灯、蓝灯的顺序循环点亮; 松开按键后 LED 熄灭, 实现了按键轮询控制 LED 颜色切换。
A/D 实验	原实验读取 ADC 转换结果, 并通过串口输出采样数据。	对 ADC 采样数据进行处理和显示, 可根据采样值计算对应的电压变化, 并通过串口输出更直观的结果。	旋转电位器时, 串口显示的 ADC 数值或电压值随输入模拟电压变化而变化, 验证了 A/D 转换功能。

LED 彩灯实验

1、实验目的

- (1) 熟悉 Eclipse 工具的使用
- (2) 能够导入已有工程
- (3) 能够掌握裸机实验的基本操作步骤
- (4) 通过本实验了解如何使用 C 语言编写裸机程序
- (5) 如何使用 GPIO 控制 LED 灯

2、实验环境

FS6818 开发平台

Eclipse 集成开发环境

虚拟机 Linux 操作系统

3、实验原理

下图 2.26 是底板 LED 灯的电路原理图。

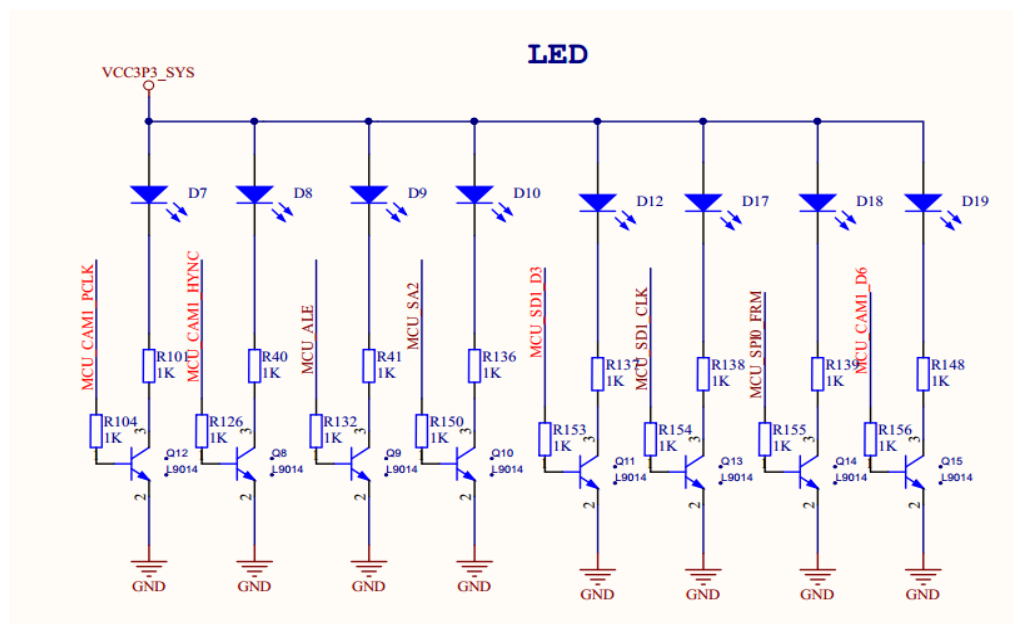


图 2.26 底板 LED 灯的电路原理图

通过如上的底板 LED 连接原理分析得知，灯的亮灭是由三极管是否导通控制的，当 MCU_CAM1_PCLK 为高电平时三极管导通，发光二极管 D7 导通，D7 灯亮，相反则灭。同理，那么此时我们只需要控制相应的 IO 引脚输出高低

电平即可控制灯的亮灭。

底板引脚	核心板引脚
MCU_CAM1_PCLK	GPIOA_28
MCU_CAM1_HYNC	GPIOE_13
MCU_ALE	GPIOB_12
MCU_SA2	GPIOC_2
MCU_SD1_D3	GPIOD_27
MCU_SD1_CLK	GPIOD_22
MCU_SD1_SPIO_FRM	GPIOC_30
MCU_CAM1_D6	GPIOB_9

查找到相应的 GPIO 编号之后，需要找到相应的芯片手册查看该寄存器的地址及配置方式。

4、实验步骤

本文提供了相关项目实验源码，在实验中可以直接导入相应工程即可。

工程源码路径：**E:\嵌入式系统 FS6818 资料\新 7 寸屏华清远见-嵌入式 ARM 实验箱资料-I-FS6818-JTAG-gt911\程序源码\ARM 裸机实验源码\fs_rgb.tar.xz。**

将该压缩文件拷贝到 **D:share** 文件夹。再打开虚拟机文件系统 **Files**，找到 **Devices** 目录下 **Computer** 目录下的 **mnt** 文件夹，如图 2.27，图 2.28 所示，再点击 **mnt** 文件夹进入下一级目录，直到进入 **share** 文件夹下，如图 2.29 所示。在 **share** 文件夹下找到 **fs_rgb** 压缩文件，并拷贝此文件到虚拟机中 **eclipse** 工作目录下。如：**/home/linux/workspace/**目录。

拷贝好文件后，对文件再右键点击 **Extract Here** 对压缩文件进行解压，如图 2.30 所示。

（注意：如果从光盘介质直接拷贝出来的目录可能会出现文件有只读属性的情况，建议查看文件夹的只读属性是否选中，确保在未勾选的状态下）

(1) 打开 Eclipse 开发工具，在 **Project Explorer** 中导入 LED 工程

（注意：工程要放在英文路径下，不能有中文路径）



图 2.27

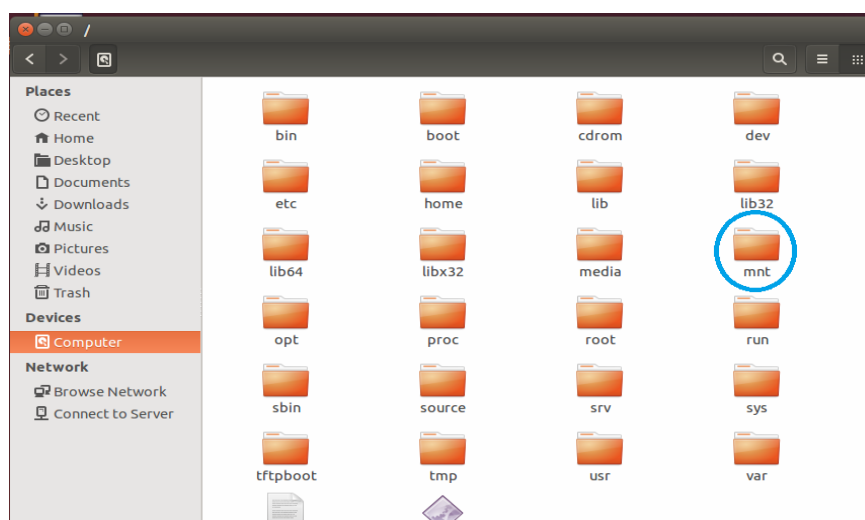


图 2.28 虚拟机中找共享文件的路径

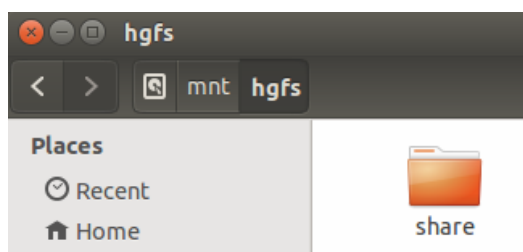


图 2.29 share 文件夹

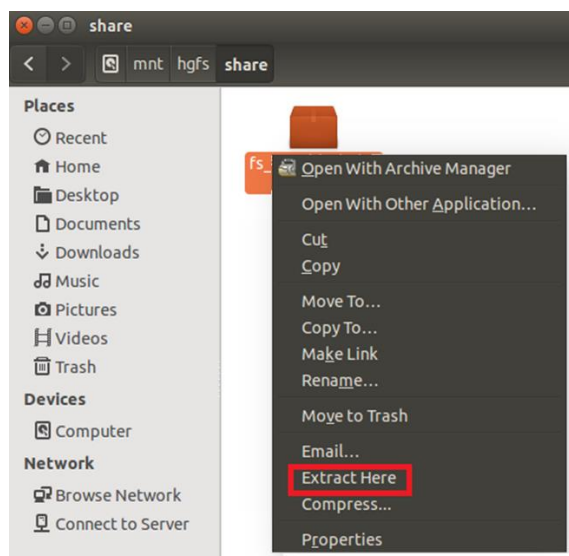


图2.30 虚拟机中文件解压

解压好文件后，在Eclipse操作界面的Project Explorer 窗口中空白处右击鼠标，选择“Import...”，如下图2.31所示。

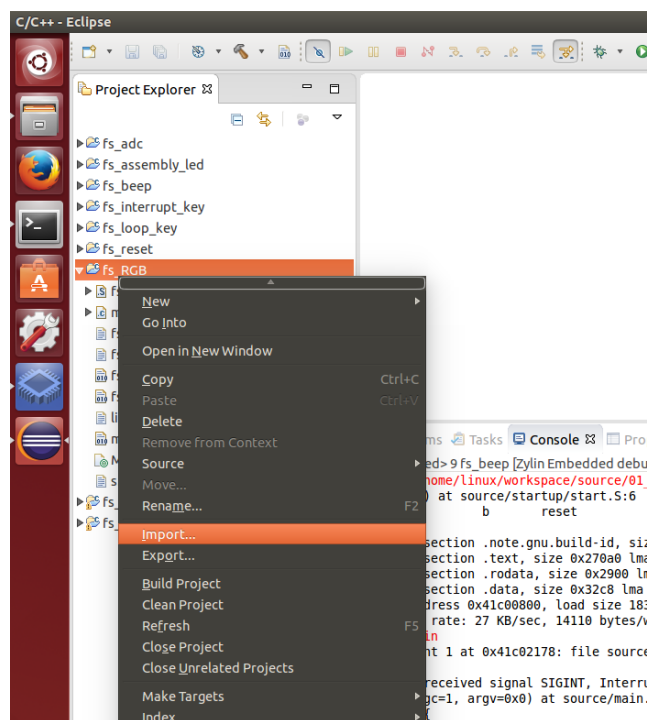


图2.31 导入界面

选择“Import...”后，出现如图2.32所示的窗口，选中C/C++下的“Existing Code as Makefile Project”，然后点击“Next”。

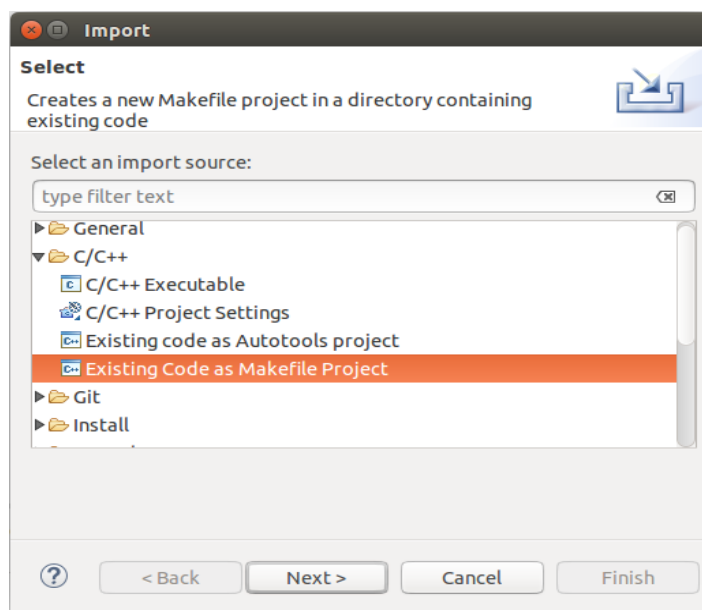


图2.32 Select选项

选择“Next”后出现如图 2.33 所示窗口，点击“Browse...”→ 出现“浏览文件夹”窗口，在“浏览文件夹”窗口中选中实验“fs_RGB”后点击“确定”

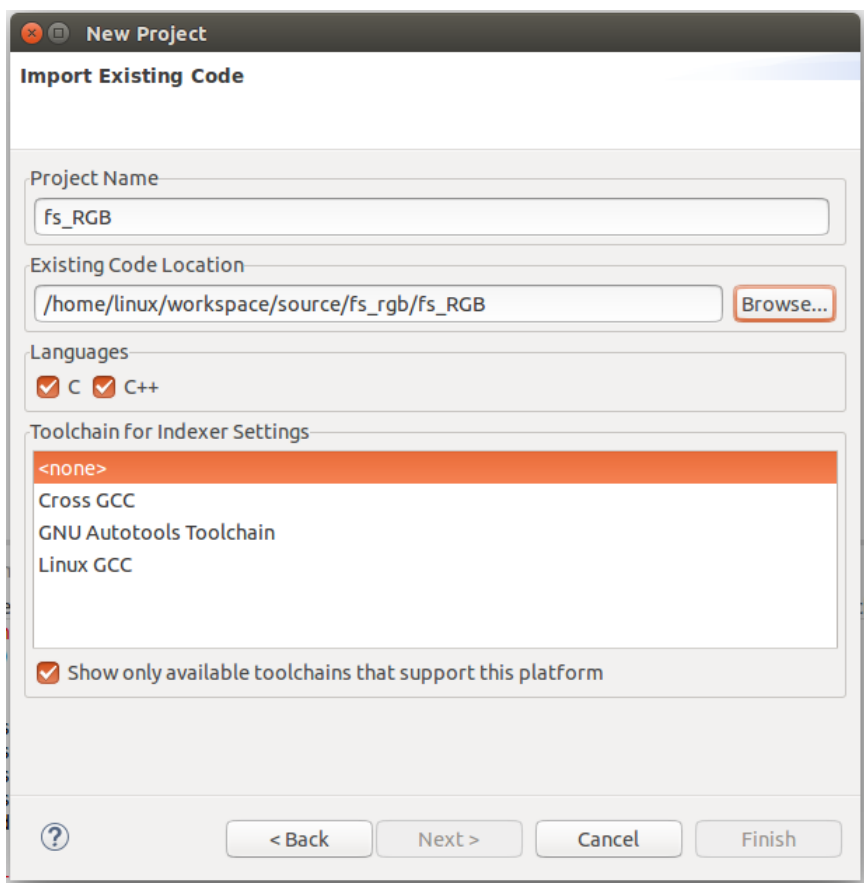


图2.33 Import Existing Code 选项

点击“确定”后出现如下所示窗口，直接点击“Finish”即可。

添加成功后可以在“Project Explorer”中看到工程成功导入。

(2) 编译工程

编译工程可以参照 2.1.2 章节操作

(3) 调试工程

调试工程可以参照 2.1.3 章节操作

5、实验代码

```
#define GPIOA28ALTFN1 (*(volatile unsigned int*)0xC001A024)

#define GPIOA28OUTENB (*(volatile unsigned int*)0xC001A004)

#define GPIOA28OUT (*(volatile unsigned int*)0xC001A000)

#define GPIOE13ALTFN0 (*(volatile unsigned int*)0xC001E020)

#define GPIOE13OUTENB (*(volatile unsigned int*)0xC001E004)

#define GPIOE13OUT (*(volatile unsigned int*)0xC001E000)

#define GPIOB12ALTFN0 (*(volatile unsigned int*)0xC001B020)
```

```
#define GPIOB12OUTENB (*(volatile unsigned int*)0xC001B004)

#define GPIOB12OUT (*(volatile unsigned int*)0xC001B000)

void rgb_init(void)
{
    unsigned int reg = 0;

    /* r */

    reg = GPIOA28ALTFN1;

    reg &= ~(0x3 << 28);

    GPIOA28ALTFN1 = reg;

    reg = GPIOA28OUTENB;

    reg &= ~(0x1 << 28); reg |= (0x1 << 28);

    GPIOA28OUTENB = reg;

    /* g */

    reg = GPIOE13ALTFN0;

    reg &= ~(0x3 << 26);

    GPIOE13ALTFN0 = reg;

    reg = GPIOE13OUTENB;

    reg &= ~(0x1 << 13);

    reg |= (0x1 << 13);

    GPIOE13OUTENB = reg;

    /* b */

    reg = GPIOB12ALTFN0;

    reg &= ~(0x3 << 24);

    reg |= (0x2 << 24);

    GPIOB12ALTFN0 = reg;

    reg = GPIOB12OUTENB;

    reg &= ~(0x1 << 12);

    reg |= (0x1 << 12);
```

```
GPIOB12OUTENB = reg;

//初始状态, 关闭所有 LED

GPIOA28OUT &= ~(0x1 << 28);
GPIOE13OUT &= ~(0x1 << 13);
GPIOB12OUT &= ~(0x1 << 12);
}

void delay(volatile int count) //延时函数
{
    while (count--);
}

void set_led(unsigned int* gpio_out, int pin, int state) //控制对应 LED 灯的亮灭
{
    if (state)
    {
        *gpio_out |= (0x1 << pin);
    }
    else
    {
        *gpio_out &= ~(0x1 << pin);
    }
}

void r_led(int state)
{
    set_led(&GPIOA28OUT, 28, state);
}
```

```
void g_led(int state)

{

    set_led(&GPIOE13OUT, 13, state);

}


void b_led(int state)

{

    set_led(&GPIOB12OUT, 12, state);

}


void main(void)

{

    rgb_init();

while (1)

{

    g_led(0);

    b_led(0);

    r_led(1);

    delay(7500000);

    g_led(1);

    b_led(0);

    r_led(0);

    delay(7500000);

    g_led(0);

    b_led(1);

    r_led(0);

    delay(7500000);

}

return;
```

6、实验现象

原实验

点击绿色箭头运行测试，



可以看到如下图 2.35 所示 LED 灯缓慢闪烁，修改延时时间可调节闪烁频率。



图2.35 缓慢闪烁的led灯

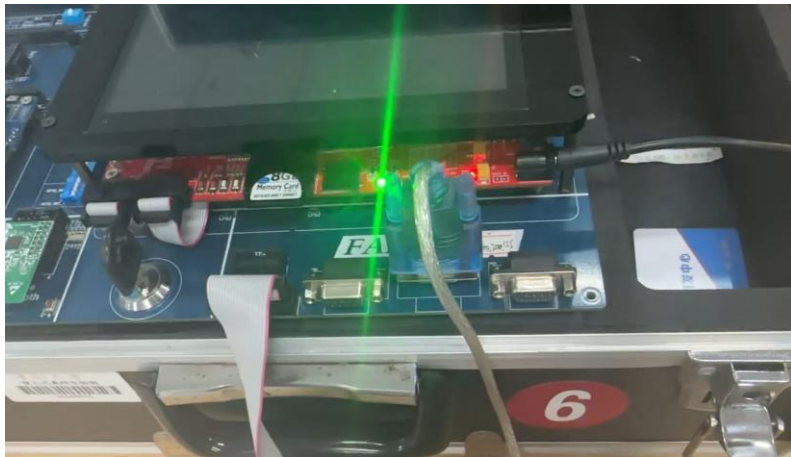
改进实验

改进后的实验中，主要改变了两个方面：**LED 闪烁频率发生变化，并且可以通过函数控制绿灯亮起。**

改进后的实验中，程序首先通过 `rgb_init()` 函数对 RGB LED 对应的 GPIO 引脚进行初始化，并将红、绿、蓝三个 LED 的初始状态设置为熄灭。程序运行后进入循环，依次调用 `r_led()`、`g_led()`、`b_led()` 函数控制不同颜色 LED 的亮灭。

实验中可以观察到，RGB LED 按照红色、绿色、蓝色的顺序依次点亮，并不断循环切换。首先红灯点亮，绿灯和蓝灯熄灭；经过一段延时后，红灯熄灭，绿灯点亮；再次延时后，绿灯熄灭，蓝灯点亮。之后程序重复上述过程，因此 LED 呈现红、绿、蓝三色循环闪烁的现象。

与原实验相比，改进实验将 LED 的控制过程封装成了 `r_led()`、`g_led()`、`b_led()` 函数，使程序结构更加清晰，也便于分别控制不同颜色的 LED。同时，改进代码将原来的双重 for 循环延时改为 `delay(7500000)` 函数延时，因此可以通过修改 `delay()` 函数的参数来调节 LED 颜色切换的快慢。当 `delay()` 参数增大时，每种颜色保持点亮的时间变长，闪烁速度变慢；当参数减小时，颜色切换速度加快。



7、心得体会

通过 LED 彩灯实验，我进一步熟悉了 FS6818 开发平台中 GPIO 外设的基本控制方法。实验过程中，需要先根据开发板底板原理图找到 LED 所对应的 GPIO 引脚，再通过查阅相关寄存器地址，对 GPIO 引脚的复用功能、输出使能和输出电平进行配置。通过这一过程，我认识到嵌入式系统中的外设控制并不是单纯依靠高级语言逻辑完成的，而是需要将程序代码与硬件电路、引脚功能以及寄存器配置结合起来分析。

在原实验中，LED 的控制主要用于验证 GPIO 输出功能是否正常。通过控制 GPIO 输出高低电平，可以观察到 LED 的亮灭变化。在改进实验中，我将红、绿、蓝三种颜色 LED 的控制分别封装成 `r_led()`、`g_led()`、`b_led()` 函数，使代码结构更加清晰，也提高了程序的可读性和可维护性。通过在主循环中依次调用不同颜色 LED 的控制函数，并配合延时函数，可以实现 RGB LED 按照红、绿、蓝顺序循环闪烁的效果。

通过本实验，我体会到函数封装在嵌入式程序设计中的重要作用。将重复的控制逻辑写成独立函数，不仅可以减少代码冗余，还便于后续功能扩展。例如，如果以后需要改变 LED 的点亮顺序或闪烁频率，只需要修改主循环逻辑或延时参数即可。同时，本实验也让我更加明确了 GPIO 控制的一般流程，即先配置引脚功能，再设置输出方向，最后通过改变输出寄存器的值来控制外设状态。

蜂鸣器实验

1、实验目的

- (1) 熟悉 Eclipse 工具的使用
- (2) 能够导入已有工程
- (3) 能够掌握裸机实验的基本操作步骤
- (4) 通过本实验了解如何使用 C 语言编写裸机程序
- (5) 如何使用 pwm 的方式控制蜂鸣器

2、实验环境

FS6818 开发平台

Eclipse 集成开发环境

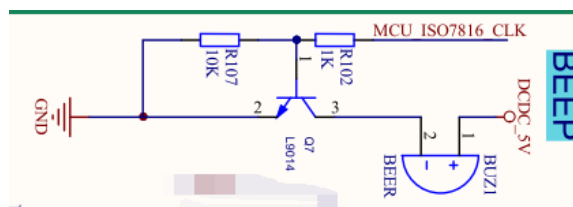
虚拟机 Linux 操作系统

3、实验原理

一般的蜂鸣器分为有源和无源，这里的“源”不是指电源，而是指震荡源。有源蜂鸣器内部自带震荡源，理论上通电就可以发声，所以可以通过 PWM 波和简单的 IO 引脚的直流信号两种方式控制；无源蜂鸣器内部不带震荡源，普通的 IO 引脚的直流信号无法令其发声，必须使用 PWM 波形才能令其发声。

FS6818 平台板载了一个有源蜂鸣器，有源蜂鸣器有内部驱动电路，但也可以通过 PWM 波驱动蜂鸣器工作。在实例中，驱动方式为通过 PWM 控制。通过控制 pwm 输出不同的波形来调节蜂鸣器的响度。

硬件连接如下：



蜂鸣器硬件连接图

底板引脚	核心板引脚
MCU_ISO7816_CLK	PWM2

编程人员通过配置 pwm2 输出方波即可。

4、实验步骤

这里的软件工具使用 putty 或者 UartAssist 均可。

拷贝路径为 E:\嵌入式系统 FS6818 资料\新 7 寸屏华清远见-嵌入式 ARM 实验箱资料-I-FS6818-JTAG-gt911\程序源码\ARM 裸机实验源码\fs_beep.tar.xz 压缩文件到 D 盘 share 文件夹下，再点击虚拟机文件系统图标 **Files**，找到 Devices 目录下 **Computer** 目录下的 **mnt** 文件夹，如第 2.2.1 节图 2.27，图 2.28 所示，再点击 mnt 文件夹进入下一级目录，直到进入 **share** 文件夹下，如图 2.29 所示。在 share 文件夹下找到 **fs_beep.tar.xz** 压缩文件，并拷贝此文件到虚拟机中 eclipse 工作目录下。如：/home/linux/workspace/目录。

拷贝好文件后，对文件再右键点击 Extract Here 对压缩文件进行解压，如图 2.30 所示。

或者使用如下命令拷贝、解压缩 tar.xz 源码文件。

```
linux@ubuntu:~$ cd ~/workspace
linux@ubuntu:~/workspace$ cp /mnt/hgfs/share/fs_beep.tar.xz ./
linux@ubuntu:~/workspace$ tar -xvf fs_beep.tar.xz
```

(1) 打开 Eclipse 开发工具，在 Project Explorer 中导入 fs_beep 工程

(注意：工程要放在英文路径下，不能有中文路径)

导入操作可以参照 2.2.1 汇编点灯实验中的第 5 点实验步骤中的导入已有工程操作，如图 2.31~2.33 所示。

(2) 编译工程

编译工程可以参照 2.1.2 章节操作

(3) 调试工程

调试工程可以参照 2.1.3 章节操作

5、实验代码

```
#include <main.h>

#define PWM_CH 2

#define TO_PERIOD_NS(freq) (1000000000UL / (freq))

#define TO_DUTY_NS(duty, freq) (TO_PERIOD_NS(freq) * (duty) / 100)
```

```
//音符频率, 单位: Hz

#define NOTE_REST 0

#define NOTE_C4 262

#define NOTE_D4 294

#define NOTE_E4 330

#define NOTE_F4 349

#define NOTE_G4 392

#define NOTE_A4 440

#define NOTE_B4 494

#define NOTE_C5 523


static void do_system_initial(void)

{

    malloc_init();

    s5p6818_pwm_init();

    s5p6818_serial_initial();

}


static void delay_ms(unsigned int ms)

{

    volatile unsigned int i, j;

    for(i = 0; i < ms; i++)

    {

        for(j = 0; j < 50000; j++)

        {

        }

    }

}
```

```
    }  
}  
  
static void play_tone(unsigned int freq, unsigned int duration_ms)  
{  
    unsigned int period;  
    unsigned int duty;  
  
    if(freq == NOTE_REST)  
    {  
        pwm_config_u(PWM_CH, 0, TO_PERIOD_NS(1000));  
        delay_ms(duration_ms);  
        return;  
    }  
  
    period = TO_PERIOD_NS(freq);  
    duty = period / 2;  
  
    pwm_config_u(PWM_CH, duty, period);  
  
    delay_ms(duration_ms);  
  
    pwm_config_u(PWM_CH, 0, period);  
    delay_ms(30);  
}  
  
//小星星前两句  
static const unsigned int melody[] = {  
    NOTE_C4, NOTE_C4, NOTE_G4, NOTE_G4,  
    NOTE_A4, NOTE_A4, NOTE_G4,
```

```
NOTE_F4, NOTE_F4, NOTE_E4, NOTE_E4,
NOTE_D4, NOTE_D4, NOTE_C4
};

static const unsigned int duration[] = {
    40, 40, 40, 40,
    40, 40, 80,

    40, 40, 40, 40,
    40, 40, 80
};

static void play_music(void)
{
    int i;
    int count;

    count = sizeof(melody) / sizeof(melody[0]);

    for(i = 0; i < count; i++)
    {
        play_tone(melody[i], duration[i]);
    }

    pwm_config_u(PWM_CH, 0, TO_PERIOD_NS(1000));
}

int main(int argc, char * argv[])
{
```

```
do_system_initial();

s5p6818_serial_write_string(0, "s5p6818 pwm music test ...\r\n");

pwm_init(PWM_CH, 0, 0);

s5p6818_pwm_enable();

play_music();

while(1)
{

}

return 0;
}
```

6、实验现象

原实验

点击左侧的绿色箭头进行测试，可以听到蜂鸣器鸣响。

改进实验

改进后的实验中，主要改变了 PWM 的输出方式，使蜂鸣器由原来发出固定频率的单一声音，变为按照不同音符频率播放音乐。

程序运行后，串口会输出：

s5p6818 pwm music test ...

随后 PWM2 开始输出 PWM 波形，蜂鸣器按照程序中设置的音符数组依次发声，实验中可以听到蜂鸣器不再是一直发出同一个音调，而是发出高低不

同、有节奏变化的声音。

改进后的程序通过 `melody[]` 数组保存小星星前两句的音符，通过 `duration[]` 数组控制每个音符的持续时间。因此，实验现象表现为蜂鸣器快速播放一次《小星星》前两句旋律。

同时，程序中每播放完一个音符后，会将 PWM 占空比设置为 0，并进行短暂延时，使相邻音符之间产生间隔。这样可以让旋律更加清楚，不会变成连续不断的一长串声音。播放完成后，程序将 PWM 占空比设置为 0，蜂鸣器停止发声，然后程序进入 `while(1)` 空循环，不会再次播放音乐。



7、心得体会

在蜂鸣器实验，我学习了 PWM 控制方式在嵌入式系统中的应用。蜂鸣器实验与普通 GPIO 控制不同，它不是简单地输出高低电平，而是通过 PWM 输出具有一定频率和占空比的方波信号，使蜂鸣器产生声音。通过本实验，我理解了 PWM 信号的基本特点：频率会影响蜂鸣器发声的音调，占空比和持续时间会影响蜂鸣器的声音效果。

蜂鸣器主要实现固定频率的鸣响，用于验证 PWM 输出是否正常。在改进实验中，我设置了不同音符对应的频率，并使用数组保存旋律和音符持续时间，使蜂鸣器能够按照设定顺序播放《小星星》前两句旋律。相比原实验中单一的蜂鸣声，改进后的实验现象更加直观，也更能体现 PWM 控制的灵活性。每个音符播放结束后，程序会将 PWM 占空比设置为 0，并加入短暂延时，使

相邻音符之间具有一定间隔，从而让旋律听起来更加清楚。

PWM 是嵌入式系统中非常常用的一种控制方式，不仅可以用于蜂鸣器发声，还可以用于 LED 调光、电机调速、舵机控制等场景。实验也让我体会到，外设控制不仅要关注能否运行，还要根据实际现象不断调整参数。例如蜂鸣器的音符持续时间如果过短，旋律就会播放得过快，不容易听清；如果延时时间过长，节奏又会变慢。因此，在嵌入式实验中，程序参数的设置需要结合实际硬件现象进行调试。

串口输出实验

1、实验目的

熟悉 Eclipse 工具的使用
能够导入已有工程
能够掌握裸机实验的基本操作步骤
通过本实验了解如何使用 C 语言编写裸机程序
了解串口的的基本配置方法

2、实验环境

FS6818 开发平台
Eclipse 集成开发环境
虚拟机 Linux 操作系统

3、实验原理

串口是开发板和开发人员之间进行交互必不可少的，所以用户需要了解如何初始化串口，如何配置串口能够让串口打印出用户需要的数据，该实验通过配置串口，可以使串口正常输出。

串口是计算机上一种非常通用设备通信的协议（不要与通用串行总线 Universal Serial Bus 或者 USB 混淆）。大多数计算机包含两个基于 RS232 的串口。串口同时也是仪器仪表设备通用的通信协议；很多 GPIB(通用接口总线)兼容的设备也带有 RS-232 口。同时，串口通信协议也可以用于获取远程采集设备的数据。

串口通信的概念非常简单，串口按位（bit）发送和接收字节。尽管比按字节（byte）的并行通信慢，但是串口可以在使用一根线发送数据的同时用另一根线接收数据。它很简单并且能够实现远距离通信。典型地，串口用于 ASCII 码字符的传输。通信使用 3 根线完成：（1）地线，（2）发送，（3）接收。由于串口通信是异步的，端口能够在在一根线上发送数据同时在另一根线上接收数据。其他线用于握手，但是不是必须的。串口通信最重要的参数是波特率、数据位、停止位和奇偶校验。

对于两个进行通行的端口，这些参数必须匹配：

波特率：这是一个衡量通信速度的参数。它表示每秒钟传送的 bit 的个数。例如 300 波特表示每秒钟发送 300 个 bit。当我们提到时钟周期时，我们就是指波特率例如如果协议需要 4800 波特率，那么时钟是 4800Hz。这意味着

串口通信在数据线上的采样率为 4800Hz。通常电话线的波特率为 14400, 28800 和 36600。波特率可以远远大于这些值, 但是波特率和距离成反比。高波特率常常用于放置的很近的仪器间的通信, 典型的例子就是 GPIB 设备的通信。

数据位: 这是衡量通信中实际数据位的参数。当计算机发送一个信息包, 实际的数据不会是 8 位的, 标准的值是 5、7 和 8 位。如何设置取决于你想传送的信息。比如, 标准的 ASCII 码是 0~127 (7 位)。扩展的 ASCII 码是 0~255 (8 位)。如果数据使用简单的文本 (标准 ASCII 码), 那么每个数据包使用 7 位数据。每个包是指一个字节, 包括开始/停止位, 数据位和奇偶校验位。由于实际数据位取决于通信协议的选取, 术语“包”指任何通信的情况。

停止位: 用于表示单个包的最后一位。典型的值为 1, 1.5 和 2 位。由于数据是在传输线上定时的, 并且每一个设备有其自己的时钟, 很可能在通信中两台设备间出现了小小的不同步。因此停止位不仅仅是表示传输的结束, 并且提供计算机校正时钟同步的机会。适用于停止位的位数越多, 不同时钟同步的容忍程度越大, 但是数据传输率同时也越慢。

奇偶校验位: 在串口通信中一种简单的检错方式。有四种检错方式: 偶、奇、高和低。当然没有校验位也是可以的。对于偶和奇校验的情况, 串口会设置校验位 (数据位后面的一位), 用一个值确保传输的数据有偶个或者奇个逻辑高位。例如, 如果数据是 011, 那么对于偶校验, 校验位为 0, 保证逻辑高的位数是偶数个。如果是奇校验, 校验位为 1, 这样就有 3 个逻辑高位。高位和低位不真正的检查数据, 简单置位逻辑高或者逻辑低校验。这样使得接收设备能够知道一个位的状态, 有机会判断是否有噪声干扰了通信或者是否传输和接收数据是否不同步。

硬件流控制: 硬件流控制常用的有 RTS/CTS 流控制和 DTR/R (数据终端就绪/数据设置就绪) 流控制。硬件流控制必须将相应的电缆线连上, 用 RTS/CTS (请求发送/清除发送) 流控制时, 应将通讯两端的 RTS、CTS 线对应相连, 数据终端设备 (如计算机) 使用 RTS 来起始调制解调器或其它数据通讯设备的数据流, 而数据通讯设备 (如调制解调器) 则用 CTS 来起动和暂停来自计算机的数据流。这种硬件握手方式的过程为: 我们在编程时根据接收端缓冲区大小设置一个高位标志 (可为缓冲区大小的 75%) 和一个低位标志 (可为缓冲区大小的 25%), 当缓冲区内数据量达到高位时, 我们在接收端将 CTS 线置低电平 (送逻辑 0), 当发送端的程序检测到 CTS 为低后, 就停止发送数据, 直到接收端缓冲区的数据量低于低位而将 CTS 置高电平。RTS 则用来标明接收设备有没有准备好接收数据。

常用的流控制还有 DTR/ R（数据终端就绪/数据设置就绪）。我们在此不再详述。

- 101 - 36600。波特率可以远远大于这些值，但是波特率和距离成反比。高波特率常常用于放置的很近的仪器间的通信，典型的例子就是 GPIB 设备的通信。

数据位：这是衡量通信中实际数据位的参数。当计算机发送一个信息包，实际的数据不会是 8 位的，标准的值是 5、7 和 8 位。如何设置取决于你想传送的信息。比如，标准的 ASCII 码是 0~127（7 位）。扩展的 ASCII 码是 0~255（8 位）。如果数据使用简单的文本（标准 ASCII 码），那么每个数据包

使用 7 位数据。每个包是指一个字节，包括开始/停止位，数据位和奇偶校验位。由于实际数据位取决于通信协议的选取，术语“包”指任何通信的情况。

S5P6818 串口控制器概述：

1. 简述

通用异步收发（UART）可支持 5 个独立的异步串行输入/输出口，每个口皆可支持中断模式及 DMA 模式(Direct Memory Access，也称为成组数据传送方式，有时也称为直接内存操作，是快速的传输模式)，UART 可产生一个中断或者发出一个 DMA 请求，来传送 CPU 与 UART 之间的数据，UART 的比特率最大可达到 4Mbps。每一个 UART 通道包含两 FIFOs（先进先出）用于数据的收发，其中通道 0 的 FIFO 大小为 256 字节，通道 1、4 的 FIFO 大小为 64 字节，通道 2、3 的 FIFO 大小为 16 字节。

2. 特点

5 组收发通道，同时支持中断模式及 DMA 操作。

通道 0、1、2 及带红外通道 3。

通道 0 带 256byte 的 FIFO，通道 1、4 带 64 byte 的 FIFO，通道 2、3 带 16byteFIFO。通道 0、1、2 支持自动流控功能，支持握手模式的发送/接收。

4、实验步骤

这里的软件工具使用 putty 或者 UartAssist 均可。

拷贝路径为 E:\嵌入式系统 FS6818 资料\新 7 寸屏华清远见-嵌入式 ARM 实验箱资料-I-FS6818-JTAG-gt911\程序源码\ARM 裸机实验源码\ fs_serial_output 压缩文件到 D 盘 share 文件夹下，再点击虚拟机文件系统图标 Files，找到 Devices 目录下 Computer 目录下的 mnt 文件夹，如第 2.2.1 节图 2.27，图 2.28 所示，再点击 mnt 文件夹进入下一级目录，直到进入 share 文件夹下，

如图 2.29 所示。在 share 文件夹下找到 fs_serial_output 压缩文件，并拷贝此文件到虚拟机中 eclipse 工作目录下。如：/home/linux/workspace/目录。

拷贝好文件后，对文件再右键点击 Extract Here 对压缩文件进行解压，如图 2.30 所示。

(1) 打开 Eclipse 开发工具，在 Project Explorer 中导入 fs_serial 工程

(注意：工程要放在英文路径下，不能有中文路径)

导入操作可以参照 2.2.1 汇编点灯实验中的第 5 点实验步骤中的导入已有工程操作，如图 2.31~2.33 所示。

(2) 编译工程

编译工程可以参照 2.1.2 章节操作

(3) 调试工程

调试工程可以参照 2.1.3 章节操作

5、实验代码

```
#include <main.h>

static void do_system_initial(void)
{
    s5p6818_serial_initial();
}

int main(int argc, char * argv[])
{
    int i = 0, j = 0;

    do_system_initial();

    /*
     * Write topic information
     */

    while(1)
```

```

{
//      s5p6818_serial_write(0, "s5p6818 serial0 stdio test ...\r\nzjt8204220813\r\n", 33);

      s5p6818_serial_write(0, " 樊博文  电信 2303 8207230322\r\n", sizeof(" 樊博文  电信 2303
8207230322\r\n") - 1);

      for ( i = 0; i < 2000; i++)

          for (j = 0; j <2000; j++);

    }

    return 0;
}
  
```

6、实验现象

原实验

点击运行按钮



可以在主机的串口调试工具中可以看到如下图 2.36 如示的打印信息。

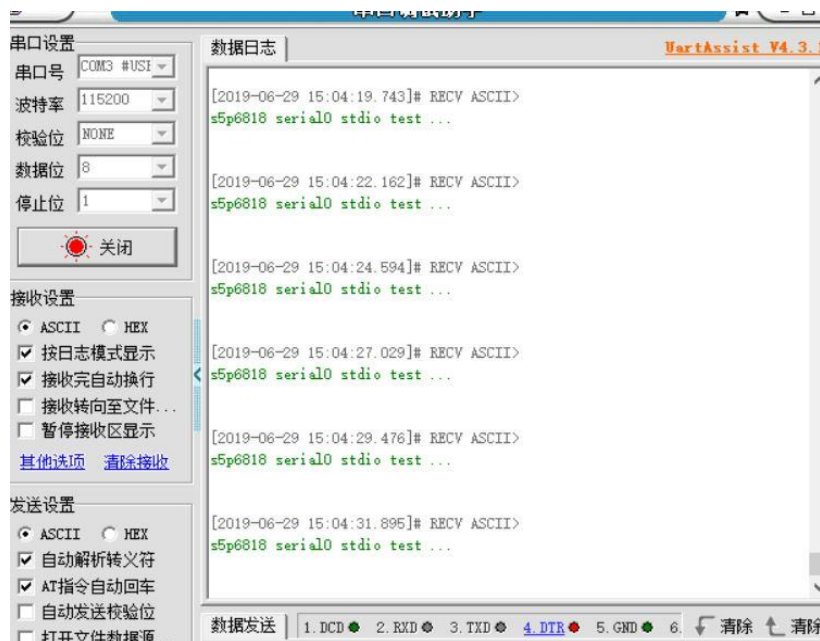


图 2.36 实现结果现象

上图 2.36 中的实验结果查看工具串口调试助手 UartAssist 可以在路径 E:\嵌入式系统 FS6818 资料\新 7 寸屏华清远见-嵌入式 ARM 实验箱资料-I-FS6818-JTAG-gt911 下如图 2.37 所示的路径中找取，在设置好这个 UartAssist 工具，

打开串口以前，需把 Putty 串口终端关闭掉，因为系统只允许一个串口输出信息。

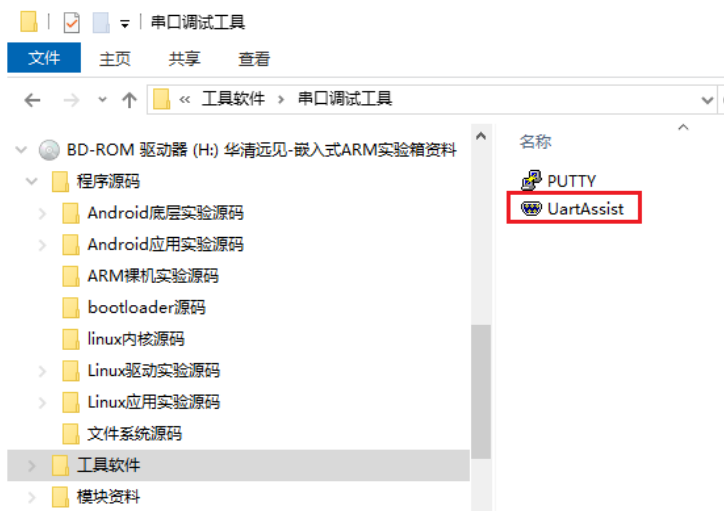


图 2.37 UartAssist 工具路径

改进实验

程序运行后，首先调用 `s5p6818_serial_initial()` 函数完成串口初始化。随后程序进入 `while(1)` 循环，通过串口 0 不断输出一行信息：

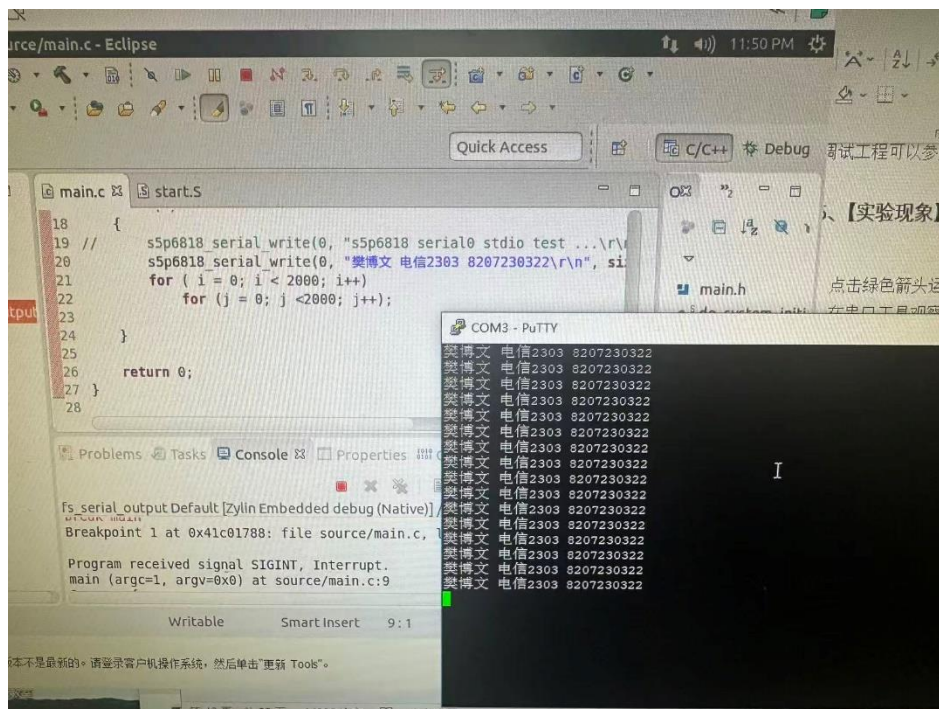
樊博文 电信 2303 8207230322

由于字符串末尾包含 `\r\n`，因此每输出一次后都会换行显示。在串口调试助手中可以观察到，该信息会按行重复打印。

每打印一次后，程序会执行一段双重 `for` 循环延时：

```
for ( i = 0; i < 2000; i++)  
for (j = 0; j < 2000; j++);
```

因此，串口信息不是瞬间连续刷屏，而是每隔一小段时间输出一次。实验现象表明，串口初始化和串口发送函数能够正常工作，开发板可以通过串口向主机发送指定的字符串信息。



7、心得体会

串口输出实验，我掌握了 UART 串口发送数据的基本方法。串口通信是嵌入式开发中最常用的调试方式之一，开发板可以通过串口将程序运行状态、变量值或提示信息发送到主机端的串口调试助手中，从而帮助开发人员判断程序是否正常运行。在本实验中，程序首先完成串口初始化，然后通过串口发送函数不断向主机输出指定字符串。

在原实验中，串口输出主要用于验证 UART 发送功能是否正常。在改进实验中，我将输出内容修改为包含姓名、班级和学号的信息，并在 while(1) 循环中反复发送，同时通过双重 for 循环加入延时，使串口信息不会瞬间连续刷屏，而是按照一定间隔重复显示。通过观察串口调试助手中的输出内容，可以判断串口初始化、数据发送以及字符串换行功能是否正常。

我认识到串口输出在嵌入式调试中的重要作用。很多时候，开发板没有复杂的显示界面，程序运行过程中的信息无法直接观察，这时串口输出就成为判断程序状态的重要手段。例如在调试 GPIO、PWM、ADC 和按键实验时，都可以通过串口输出提示信息来辅助判断程序是否执行到指定位置。同时，本实验也让我注意到字符串长度、回车换行符和发送函数参数等细节问题。如果字符串长度设置不正确，可能会导致输出内容不完整；如果没有添加 \r\n，串口显示时可能不会自动换行。因此，串口输出虽然原理简单，但在实际使用中也需要注意细节。

串口输入实验

1、实验目的

- (1) 熟悉 Eclipse 工具的使用
- (2) 能够导入已有工程
- (3) 能够掌握裸机实验的基本操作步骤
- (4) 通过本实验了解如何使用 C 语言编写裸机程序
- (5) 了解串口的基本配置方法

2、实验环境

FS6818 开发平台
Eclipse 集成开发环境
虚拟机 Linux 操作系统

3、实验原理

实验原理与串口输出实验一致。

4、实验步骤

这里的软件工具使用 putty 或者 UartAssist 均可。

拷贝路径为 E:\嵌入式系统 FS6818 资料\新 7 寸屏华清远见-嵌入式 ARM 实验箱资料-I-FS6818-JTAG-gt911\程序源码\ARM 裸机实验源码\ fs_serial_input 压缩文件到 D 盘 share 文件夹下，再从虚拟机下的文件系统图标 **Files** 中找到 Devices 目录下 Computer 目录下的 mnt 文件夹，如图 2.27，图 2.28 所示，再点击 mnt 文件夹进入下一级目录，直到进入 share 文件夹下，如图 2.29 所示。在 share 文件夹下找到 fs_serial_input 压缩文件，并拷贝此文件到虚拟机中 eclipse 工作目录下。如：/home/linux/workspace/目录。

拷贝好文件后，对文件再右键点击 Extract Here 对压缩文件进行解压，如图 2.30 所示。

(1) 打开 Eclipse 开发工具，在 Project Explorer 中导入 fs_serial_input 工程

(注意：工程要放在英文路径下，不能有中文路径)

导入操作可以参照 2.2.1 汇编点灯实验中的第 5 点实验步骤中的导入已有工程操作，如图 2.31~2.33 所示。

(2) 编译工程

编译工程可以参照 2.1.2 章节操作

(3) 调试工程

调试工程可以参照 2.1.3 章节操作

5、实验代码

```
#include <main.h>

#define BUF_SIZE 256

static void do_system_initial(void)
{
    s5p6818_serial_initial();
}

int main(int argc, char * argv[])
{
    u8_t buf[BUF_SIZE] = {0};

    u8_t ch = 0;

    int pos = 0;

    int i = 0;

    do_system_initial();

    s5p6818_serial_write_string(0, "  Serial Echo Test\r\n");

    s5p6818_serial_write_string(0, "Please type Chinese or English, then press Enter.\r\n");

    while(1)
    {
        if(s5p6818_serial_read(0, &ch, 1) == 1)
        {
```

```
if(ch == '\r' || ch == '\n')
{

    if(pos == 0)
    {
        continue;
    }

    s5p6818_serial_write_string(0, "recv: [");
    s5p6818_serial_write(0, (char *)buf, pos);
    s5p6818_serial_write_string(0, "]\r\n");

    for(i = 0; i < BUF_SIZE; i++)
    {
        buf[i] = 0;
    }

    pos = 0;
}
else
{

    if(pos < BUF_SIZE - 1)
    {
        buf[pos++] = ch;
    }
    else
    {

```

```
s5p6818_serial_write_string(0, "recv: [");  
  
s5p6818_serial_write(0, (char *)buf, pos);  
  
s5p6818_serial_write_string(0, "]r\n");  
  
  
for(i = 0; i < BUF_SIZE; i++)  
{  
  
    buf[i] = 0;  
  
}  
  
pos = 0;  
  
}  
  
}  
  
}  
  
}  
  
return 0;  
}
```

6、实验现象

原实验

点击绿色箭头运行测试



在串口工具观察到等待用户输入，即可在串口调试助手中输入单个字符，之后串口会回显出用户输入的字符。

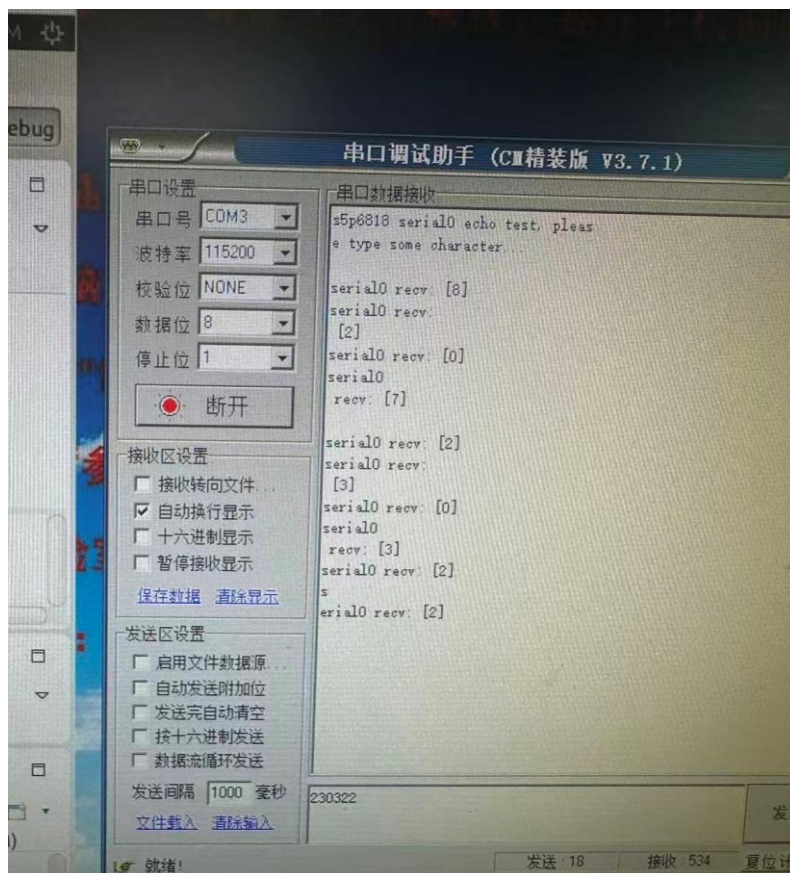


图 2.38 串口输入数据及显示

改进实验

程序编译、下载并运行后，打开串口调试助手，串口参数设置为 COM3、波特率 115200、无校验位、8 位数据位、1 位停止位。串口连接成功后，调试助手接收窗口能够显示程序输出的提示信息，说明串口初始化成功，程序已经进入等待接收数据的状态。

在原实验程序中，串口每接收到一个字节就立即进行一次处理和显示。实验时在串口调试助手的发送框中输入数字或英文字符，例如输入 我的学号 8207230322，接收窗口会依次显示每个字符对应的接收结果，如 serial0 recv [8]、serial0 recv [2]、serial0 recv [0] 等。由此可以看出，原程序能够完成串口输入数据的接收，但是它是按单个字节逐次接收和回显的。



在改进后的实验中，程序不再对每个接收到的字节立即回显，而是先将接收到的数据保存到缓冲区中。当检测到回车符或换行符后，程序认为一行数据接收完成，再将缓冲区中的内容整体回传到串口调试助手中。实验时，在发送框中输入中文字符并发送，可以在接收窗口中观察到程序能够回传中文内容。例如输入“中”“南”“大”“学”等中文字符后，串口调试助手中可以显示对应的回传结果，如：

recv: [中]

recv: [南]

recv: [大]

recv: [学]

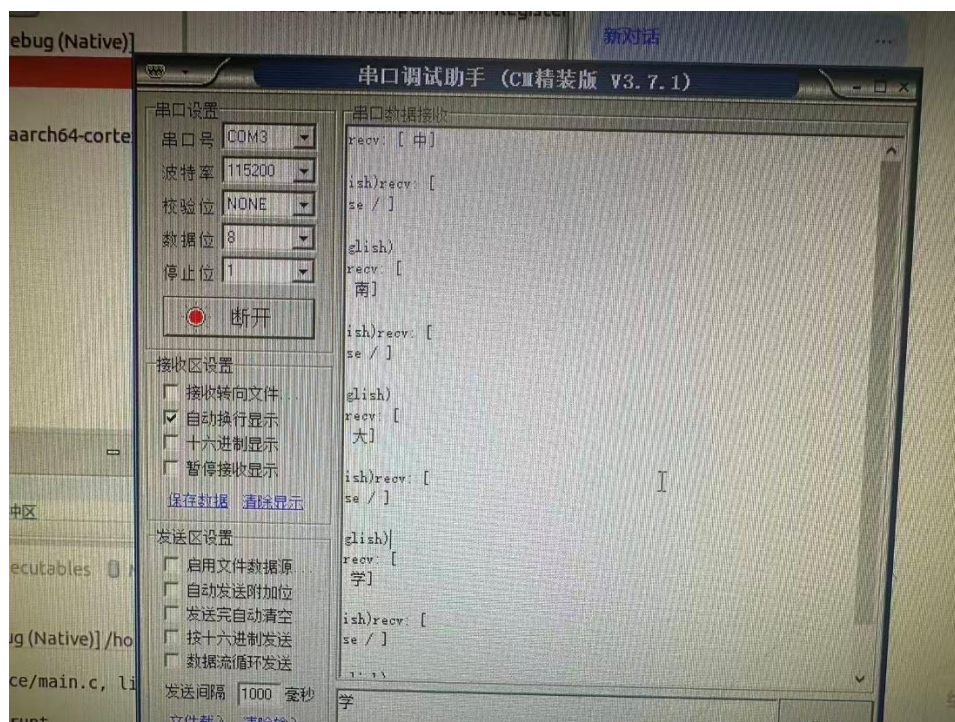
若输入英文、数字或中英文混合字符串，例如：abc 中南大学 123

串口接收窗口也能够对应显示：recv: [abc 中南大学 123]

实验结果表明，改进后的程序能够将接收到的数据先存入缓冲区，再按整行数据统一回传。由于中文字符通常由多个字节组成，采用缓冲区接收后，中文字符对应的多个字节能够被连续保存并统一发送，从而避免了按单字节

立即显示时可能出现的中文显示不完整或乱码问题。

因此，本实验实现了串口对英文、数字以及中文字符串的接收和回传功能。程序每次完成一次回传后会清空缓冲区，并继续等待下一次串口输入，说明程序能够持续进行串口接收与回显测试。



7、心得体会

串口输入实验，我进一步理解了 UART 串口接收数据的过程。与串口输出相比，串口输入不仅要求开发板能够接收来自主机的数据，还需要程序对接收到的数据进行判断、保存和处理。在原实验中，程序每接收到一个字节就立即进行回显，这种方式可以验证串口接收功能是否正常，但对于较长字符串或中文字符来说，处理方式还不够完善。

通过实验，我认识到串口通信中的数据处理方式需要根据实际应用场景进行设计。对于简单按键命令，可以逐字节判断；而对于字符串输入、中文输入或较长数据传输，则更适合采用缓冲区接收的方式。本实验也让我理解了缓冲区、数组下标、字符串结束条件等概念在嵌入式程序中的实际作用。同时，程序还需要考虑缓冲区溢出问题，当接收数据超过缓冲区大小时，应及时处理或清空缓冲区，避免数据异常。通过这个实验，我对串口接收程序的完整流程有了更加清楚的认识。

复位实验

1、实验环境

FS6818 开发平台

Eclipse 集成开发环境

虚拟机 Linux 操作系统

2、实验目的

- (1) 熟悉 Eclipse 工具的使用
- (2) 能够导入已有工程
- (3) 能够掌握裸机实验的基本操作步骤
- (4) 通过本实验了解如何使用 C 语言编写裸机程序
- (5) 了解如何通过软复位的方式控制开发板的启动

3、实验原理

如何通过程序操控 PWRCONT 和 PWRMODE 寄存器控制开发板的复位功能。

4.9.1.29 PWRCONT

- Base Address: 0xC001_0000
- Address = Base Address + 0224h, Reset Value = 0x0000_FF00

Name	Bit	Type	Description	Reset Value
RSVD	[31:16]	R	Reserved	16'h0
USE_WFI	[15:12]	RW	Use STANDBYWFI[n] signal as indicating signal to go into STOP mode.	4'hF
USE_WFE	[11:8]	RW	Use STANDBYWFE[n] signal as indicating signal to go into STOP mode.	4'hF
RSVD	[7:5]	R	Reserved	3'b000
XTAL_PWRDN	[4]	RW	X-tal power down mode selection This controls the power down of X-tal-PAD in stop-mode. 0 = XTAL is powered down in STOP mode 1 = XTAL is not powered down in STOP mode	1'b0
SWRSTENB	[3]	RW	Software Reset Enable. 0 = Disable 1 = Enable	1'b0
RSVD	[2]	RW	Reserved (This bit always should be "0")	1'b0
RTCWKENB	[1]	RW	RTC Wake-up enable 0 = Disable 1 = Enable	1'b0
RSVD	[0]	RW	Reserved (This bit always should be "0")	1'b0

4.9.1.30 PWRMODE

- Base Address: 0xC001_0000
- Address = Base Address + 0228h, Reset Value = 0x0000_0000

Name	Bit	Type	Description	Reset Value
RSVD	[31:16]	RW	Reserved	16'h0
CHGPLL	[15]	RW	Change PLL Value with new value defined in PLL Setting Register (PLL0set, PLL1set) in clock Controller. Read 0 = Stable 1 = PLL is Unstable Write 0 = None 1 = PLL Value Change	1'b0
RSVD	[14:13]	–	Reserved	2'b00
SWRST	[12]	W	This bit is cleared after Software Reset 0 = Do Not Reset 1 = Go to Reset	1'b0

4、实验步骤

这里的软件工具使用 putty 或者 UartAssist 均可。

拷贝路径为 E:\嵌入式系统 FS6818 资料\新 7 寸屏华清远见-嵌入式 ARM 实验箱资料-I-FS6818-JTAG-gt911\程序源码\ARM 裸机实验源码\ fs_reset 压缩文件到 D 盘 share 文件夹下，再从虚拟机下的文件系统图标 **Files** 中找到 Devices 目录下 Computer 目录下的 mnt 文件夹，如图 2.27，图 2.28 所示，再点击 mnt 文件夹进入下一级目录，直到进入 share 文件夹下，如图 2.29 所示。在 share 文件夹下找到 fs_reset 压缩文件，并拷贝此文件到虚拟机中 eclipse 工作目录下。如：/home/linux/workspace/目录。

拷贝好文件后，对文件再右键点击 Extract Here 对压缩文件进行解压，如图 2.30 所示。

(1) 打开 Eclipse 开发工具，在 Project Explorer 中导入 fs_reset 工程

(注意：工程要放在英文路径下，不能有中文路径)

导入操作可以参照 2.2.1 汇编点灯实验中的第 5 点实验步骤中的导入已有工程操作，如图 2.31~2.33 所示。

(2) 编译工程

编译工程可以参照 2.1.2 章节操作

(3) 调试工程

调试工程可以参照 2.1.3 章节操作

5、实验代码

```
#include <main.h>

#define GPIOA28ALTFN1 (*(volatile unsigned int*)0xC001A024)

#define GPIOA28OUTENB (*(volatile unsigned int*)0xC001A004)

#define GPIOA28OUT (*(volatile unsigned int*)0xC001A000)

#define GPIOE13ALTFN0 (*(volatile unsigned int*)0xC001E020)

#define GPIOE13OUTENB (*(volatile unsigned int*)0xC001E004)

#define GPIOE13OUT (*(volatile unsigned int*)0xC001E000)

#define GPIOB12ALTFN0 (*(volatile unsigned int*)0xC001B020)

#define GPIOB12OUTENB (*(volatile unsigned int*)0xC001B004)

#define GPIOB12OUT (*(volatile unsigned int*)0xC001B000)

void rgb_init(void)

{

unsigned int reg = 0;

/* r */

reg = GPIOA28ALTFN1;

reg &= ~(0x3 << 28);

GPIOA28ALTFN1 = reg;

reg = GPIOA28OUTENB;

reg &= ~(0x1 << 28); reg |= (0x1 << 28);

GPIOA28OUTENB = reg;

/* g */

reg = GPIOE13ALTFN0;

reg &= ~(0x3 << 26);

GPIOE13ALTFN0 = reg;

reg = GPIOE13OUTENB;

reg &= ~(0x1 << 13);

reg |= (0x1 << 13);
```

```
GPIOE13OUTENB = reg;

/* b */

reg = GPIOB12ALTFN0;
reg &= ~(0x3 << 24);
reg |= (0x2 << 24);
GPIOB12ALTFN0 = reg;
reg = GPIOB12OUTENB;
reg &= ~(0x1 << 12);
reg |= (0x1 << 12);
GPIOB12OUTENB = reg;

GPIOA28OUT &= ~(0x1 << 28);
GPIOE13OUT &= ~(0x1 << 13);
GPIOB12OUT &= ~(0x1 << 12);
}

static void do_system_initial(void)
{
    malloc_init();
    s5p6818_serial_initial();
}

void set_led(unsigned int* gpio_out, int pin, int state) //控制对应 LED 灯的亮灭
{
    if (state)
    {
        *gpio_out |= (0x1 << pin);
    }
    else
    {

```

```
*gpio_out &= ~(0x1 << pin);

}

}

void r_led(int state)
{
    set_led(&GPIOA28OUT, 28, state);
}

void g_led(int state)
{
    set_led(&GPIOE13OUT, 13, state);
}

void b_led(int state)
{
    set_led(&GPIOB12OUT, 12, state);
}

int main(int argc, char * argv[])
{
    volatile int i = 0;

    volatile int j = 0;

    do_system_initial();

    rgb_init();

    s5p6818_serial_write_string(0, "S5P6818 reset test...\r\n");

    s5p6818_serial_write_string(0, "System will reset\r\n");
```

```
s5p6818_serial_write_string(0, "Wait for system restart\r\n");

g_led(0);

b_led(0);

r_led(1);

for (i = 0; i < 2000; i++)
    for (j = 0; j < 3000; j++);

write32((S5P6818_SYS_PWRCONT), (read32((S5P6818_SYS_PWRCONT)) & ~(0x1<<3)) | (0x1<<3));
write32((S5P6818_SYS_PWRMODE), (read32((S5P6818_SYS_PWRMODE)) & ~(0x1<<12)) | (0x1<<12));

while(1) {
    s5p6818_serial_write_string(0, "It's not going to print!!!!\r\n");

}

return 0;
}
```

6、实验现象

原实验

点击绿色箭头运行测试，



可以在串口工具中看到重置倒计时，当倒计时为 0 后系统重启。

```

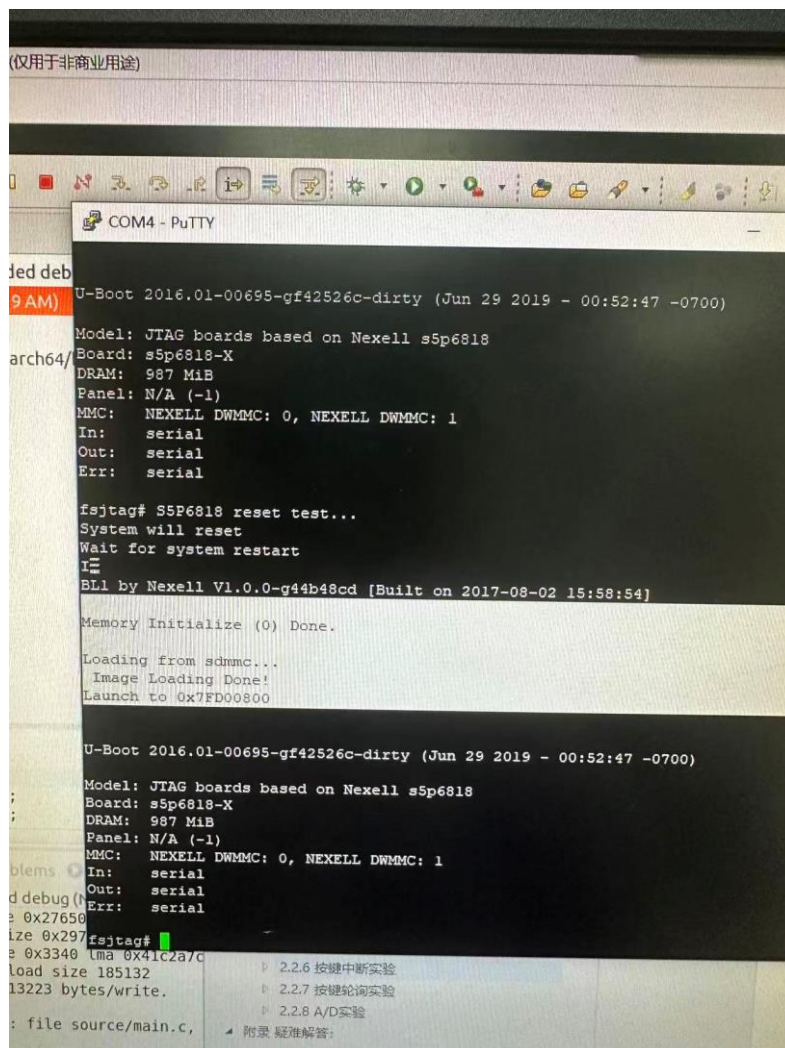
[2019-06-29 15:41:36.565]# RECV ASCII>
S5P6818 reset test...
System will reset
Wait for system restart

[2019-06-29 15:41:39.787]# RECV ASCII>
I□
BL1 by Nexell V1.0.0-g44b48cd [Built on 2017-08-02 15:58:54]

[2019-06-29 15:41:39.896]# RECV ASCII>
Memory Initialize (0) Done.

Loading from sdmmc...
Image Loading Done!
  
```

图 2.39 串口终端显示复位信息



改进实验

程序运行后，串口调试助手中首先输出复位测试提示信息：

S5P6818 reset test...

System will reset

Wait for system restart

随后程序点亮红色 LED，并经过一段延时后，通过写入系统电源控制寄存器触发 S5P6818 软件复位。复位发生后，开发板重新启动，因此串口调试助手中继续显示 U-Boot 启动信息，例如：

U-Boot 2016.01 ...

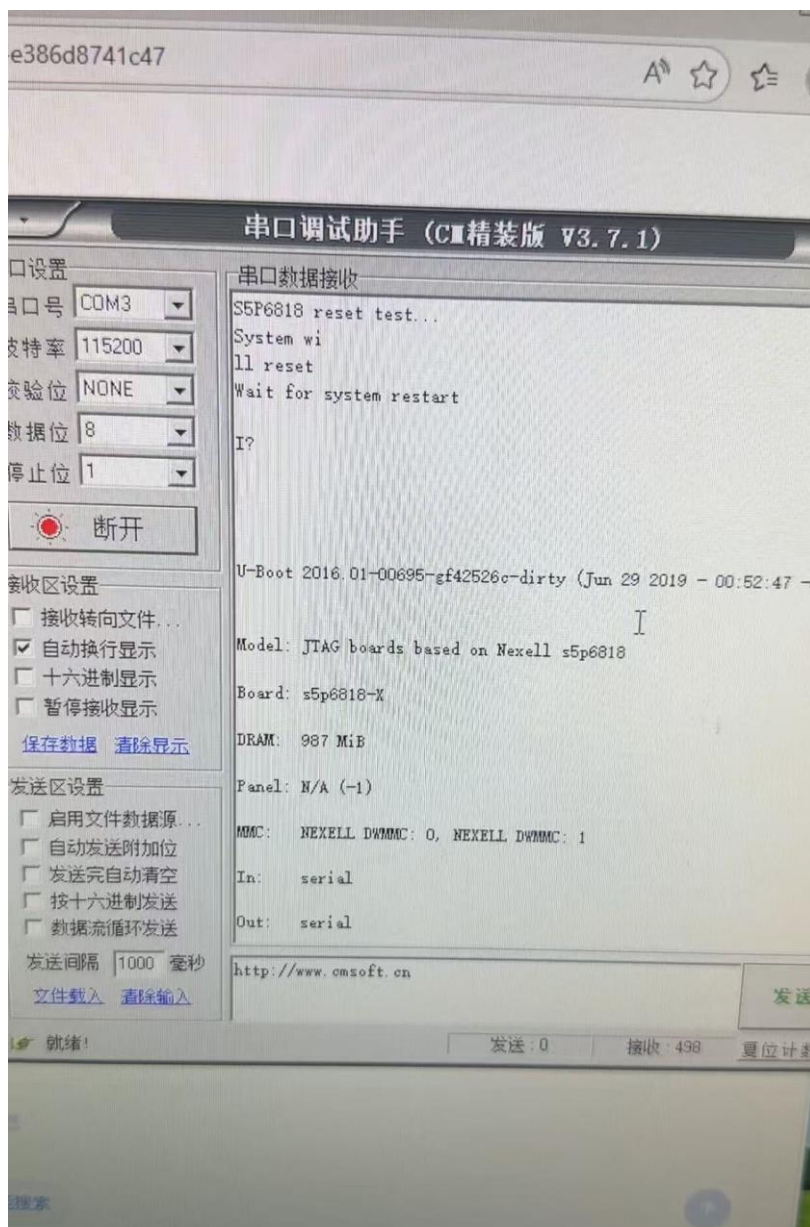
Model: JTAG Boards based on Nexell s5p6818

Board: s5p6818-X

DRAM: 987 MiB

这说明系统已经不是继续执行原来的程序，而是发生了复位并重新进入启动流程。





7、心得体会

通过复位实验，我了解了嵌入式系统中软件复位的基本实现方式。复位是嵌入式系统中非常重要的功能，当系统运行异常或需要重新启动时，可以通过硬件复位或软件复位使系统重新进入启动流程。本实验通过程序配置系统电源控制相关寄存器，使开发板产生软件复位。复位发生后，系统不再继续执行原来的程序，而是重新启动，并在串口终端中显示 U-Boot 启动信息。

原实验，程序主要通过串口输出倒计时或复位提示信息，随后触发系统复位。在改进实验中，我在复位前增加了红色 LED 状态指示，使复位前的程序执行过程更加直观。程序运行后，串口先输出复位提示信息，同时红灯点亮，经过一段

延时后触发软件复位。通过观察串口中重新出现的启动信息，可以判断系统已经成功复位。

通过这次实验，我认识到复位并不是普通意义上的程序跳转，而是系统状态的重新初始化。复位发生后，CPU、外设和程序运行环境都会重新进入启动状态，因此复位语句后面的代码通常不会继续执行。实验中写在复位操作之后的串口输出语句一般不会被打印出来，这也进一步说明复位已经成功触发。通过该实验，我对嵌入式系统启动流程、软件复位机制以及串口观察复位现象的方法有了更加直观的理解。

按键中断实验

1、实验环境

FS6818 开发平台
Eclipse 集成开发环境
虚拟机 Linux 操作系统

2、实验目的

熟悉 Eclipse 工具的使用
能够导入已有工程
能够掌握裸机实验的基本操作步骤
通过本实验了解如何使用 C 语言编写裸机程序
了解 ARM 异常及中断处理

3、实验原理

本实验中通过按键中断实现 rgb 灯的点亮，首先需要将按键 GPIO 引脚配置为中断模式，并设置中断触发方式，分别使能中断控制器及中断源。如下图 2.40 所示为实验两按键原理图，分别为 GPIOB_8 和 GPIOB_16。

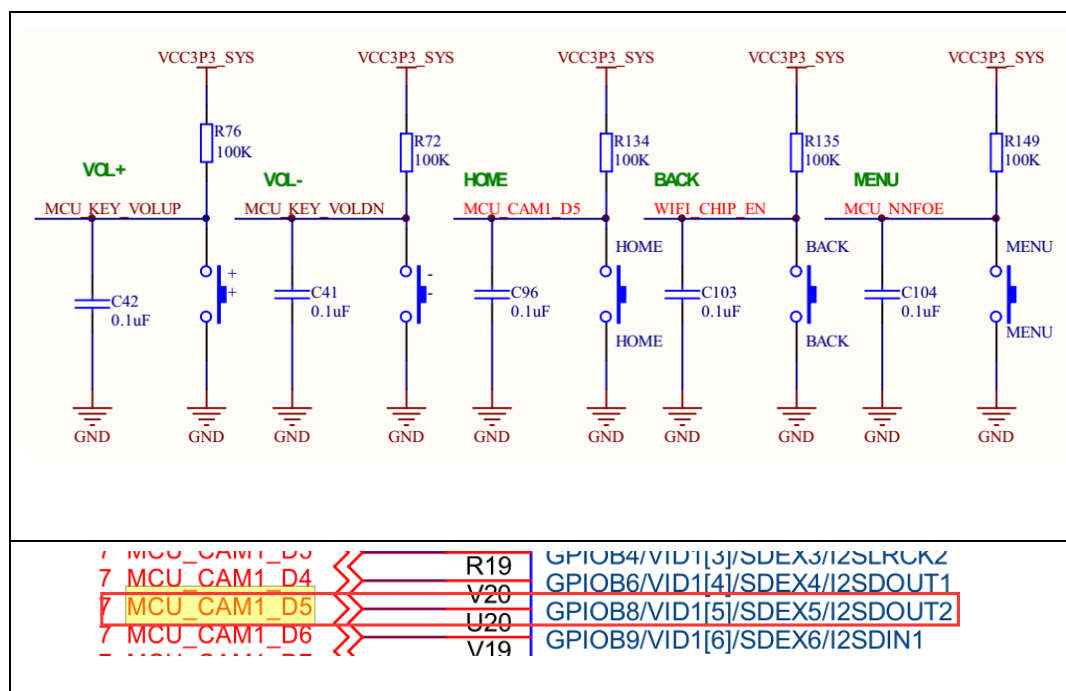


图 2.40 按键原理图

4、实验步骤

这里的软件工具使用 putty 或者 UartAssist 均可。

拷贝路径为 E:\嵌入式系统 FS6818 资料\新 7 寸屏华清远见-嵌入式 ARM 实验箱资料-I-FS6818-JTAG-gt911\程序源码\ARM 裸机实验源码\fs_interrupt_key 压缩文件到 D 盘 share 文件夹下，再从虚拟机下的文件系统图标 **Files** 中找到 Devices 目录下 Computer 目录下的 mnt 文件夹，如图 2.27，图 2.28 所示，再点击 mnt 文件夹进入下一级目录，直到进入 share 文件夹下，如图 2.29 所示。在 share 文件夹下找到 fs_interrupt_key 压缩文件，并拷贝此文件到虚拟机中 eclipse 工作目录下。如：/home/linux/workspace/目录。

拷贝好文件后，对文件再右键点击 Extract Here 对压缩文件进行解压，如图 2.30 所示。

(1) 打开 Eclipse 开发工具，在 Project Explorer 中导入 fs_serial 工程

(注意：工程要放在英文路径下，不能有中文路径)

导入操作可以参照 2.2.1 汇编点灯实验中的第 5 点实验步骤中的导入已有工程操作，如图 2.31~2.33 所示。

(2) 编译工程

编译工程可以参照 2.1.2 章节操作

(3) 调试工程

调试工程可以参照 2.1.3 章节操作

5、实验代码

```
#include <main.h>

volatile int state = 0;

// LED 寄存器定义

#define GPIOA28ALTFN1 (*(volatile unsigned int*)0xC001A024)

#define GPIOA28OUTENB (*(volatile unsigned int*)0xC001A004)

#define GPIOA28OUT      (*(volatile unsigned int*)0xC001A000)

#define GPIOE13ALTFN0 (*(volatile unsigned int*)0xC001E020)

#define GPIOE13OUTENB (*(volatile unsigned int*)0xC001E004)
```

```
#define GPIOE13OUT    (*(volatile unsigned int*)0xC001E000)

#define GPIOB12ALTFN0 (*(volatile unsigned int*)0xC001B020)

#define GPIOB12OUTENB (*(volatile unsigned int*)0xC001B004)

#define GPIOB12OUT     (*(volatile unsigned int*)0xC001B000)


#define KEY_ALTFN      (*(volatile unsigned int*)0xC001B028) // GPIOBALTFN1
#define KEY_OUTENB     (*(volatile unsigned int*)0xC001B004) // GPIOBOUTENB
#define KEY_DETMODE    (*(volatile unsigned int*)0xC001B00C) // GPIOBDETMODE
#define KEY_PAD        (*(volatile unsigned int*)0xC001B008) // GPIOBPAD


void rgb_init(void)
{
    unsigned int reg = 0;

    /* r - GPIOA28 */
    reg = GPIOA28ALTFN1;
    reg &= ~(0x3 << 28);
    GPIOA28ALTFN1 = reg;
    reg = GPIOA28OUTENB;
    reg |= (0x1 << 28);
    GPIOA28OUTENB = reg;

    /* g - GPIOE13 */
    reg = GPIOE13ALTFN0;
    reg &= ~(0x3 << 26);
    GPIOE13ALTFN0 = reg;
    reg = GPIOE13OUTENB;
    reg |= (0x1 << 13);
    GPIOE13OUTENB = reg;
```

```
/* b - GPIOB12 */

reg = GPIOB12ALTFN0;

reg &= ~(0x3 << 24);

reg |= (0x1 << 24);

GPIOB12ALTFN0 = reg;

reg = GPIOB12OUTENB;

reg |= (0x1 << 12);

GPIOB12OUTENB = reg;


GPIOA28OUT &= ~(0x1 << 28);

GPIOE13OUT &= ~(0x1 << 13);

GPIOB12OUT &= ~(0x1 << 12);
}

// 按键初始化为外部中断模式

void key_interrupt_init(void)
{
    unsigned int reg = 0;

    // 配置 GPIOB30 为外部中断模式 (ALT FUNCTION 0xF)

    reg = KEY_ALTFN;

    reg &= ~(0x3 << 28);

    reg |= (0xF << 28);

    KEY_ALTFN = reg;


    // 禁用输出

    reg = KEY_OUTENB;

    reg &= ~(0x1 << 30);

    KEY_OUTENB = reg;
```

```
// 设置上升沿检测

reg = KEY_DETMODE;

reg &= ~(0x3 << 28);

reg |= (0x1 << 28);

KEY_DETMODE = reg;
}

void delay(volatile int count)
{
    while (count--);
}

void set_led(unsigned int* gpio_out, int pin, int val)
{
    if (val)
        *gpio_out |= (0x1 << pin);
    else
        *gpio_out &= ~(0x1 << pin);
}

void r_led(int val) { set_led(&GPIOA28OUT, 28, val); }
void g_led(int val) { set_led(&GPIOE13OUT, 13, val); }
void b_led(int val) { set_led(&GPIOB12OUT, 12, val); }

static void do_system_initial(void)
{
    malloc_init();

    s5p6818_irq_init();

    s5p6818_serial_initial();
}
```

```
key_initial();
}

int main(int argc, char * argv[])
{
    volatile int i = 0;

    do_system_initial();

    rgb_init();

    key_interrupt_init();

    s5p6818_irq_set_type(128 + 8, IRQ_TYPE_EDGE_RISING);

    s5p6818_irq_enable(128 + 8);

    r_led(0);
    g_led(0);
    b_led(0);

    // 初始 state 为 0

    state = 0;

    s5p6818_serial_write_string(0, "System Ready. All LEDs OFF.\r\n");
    s5p6818_serial_write_string(0, "Press button to light Red LED...\r\n");

    while(1)
    {
        if (state == 1)
        {
            state = 0;
```

```
s5p6818_serial_write_string(0, "Button Pressed! Red LED ON\r\n");

// 红灯亮

r_led(1);

for (i = 0; i < 500000; i++);

r_led(0);

s5p6818_serial_write_string(0, "Red LED OFF\r\n");

}

for (i = 0; i < 1000; i++);

}

return 0;

}
```

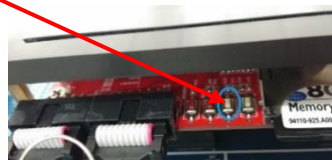
6、实验现象

原实验

点击运行按钮



在串口终端会打印内容，按下中断按钮，右起第 2 个：



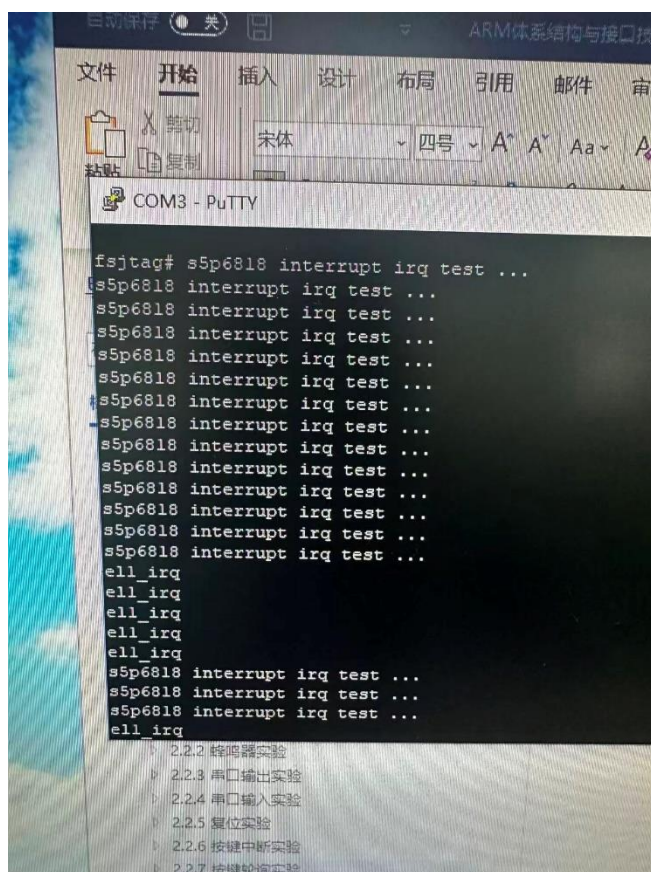
串口终端就会出现与之前没按键时不同的内容，如图 2.41 所示。

COM3 - PuTTY

```
Board: s5p6818-X
DRAM: 987 MiB
Panel: N/A (-1)
MMC: NEXELL DWMMC: 0, NEXELL DWMMC: 1
In: serial
Out: serial
Err: serial

fsjtag# s5p6818 interrupt irq test ...
s5p6818 interrupt irq test ...
s5p6818 interrupt irq test ...
s5p6818 interrupt irq test ...
s5p6818 interrupt irq test ...
s5p6818 interrupt irq test ...
ell_irq
ell_irq
ell_irq
ell_irq
s5p6818 interrupt irq test ...
s5p6818 interrupt irq test ...
```

图 2.41 串口终端出结果



改进实验

程序运行后，系统首先完成内存、中断、串口和按键的初始化，然后初始化 RGB LED 对应的 GPIO 引脚，并将红灯、绿灯、蓝灯全部设置为熄灭状态。

串口调试助手中会先显示：

System Ready. All LEDs OFF.

Press button to light Red LED...

此时开发板上的 RGB LED 全部处于熄灭状态，程序进入循环等待按键中断。

当按下按键后，按键引脚 GPIOB30 产生中断，中断服务程序将 state 变量置为 1。主程序检测到 `state == 1` 后，会在串口输出：

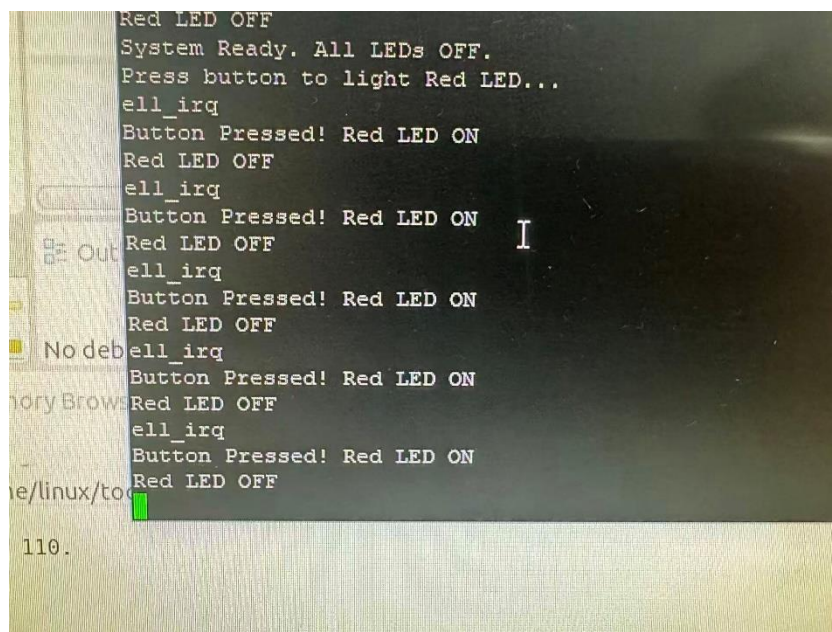
Button Pressed! Red LED ON

同时点亮红色 LED，红灯保持点亮一小段时间后熄灭，随后串口继续输出：

Red LED OFF

整个过程中，绿色 LED 和蓝色 LED 一直保持熄灭状态。之后程序继续等待下一次按键触发。当再次按下按键时，红灯会再次短暂亮起并熄灭，串口也会再次打印对应提示信息。





7、心得体会

通过按键中断实验，我学习了外部中断的基本原理和处理流程。在嵌入式系统中，按键是一种常见的输入设备。相比轮询方式，中断方式不需要 CPU 一直检查按键状态，而是在按键事件发生时由硬件主动触发中断请求，CPU 再转去执行对应的中断处理程序。因此，中断方式具有响应及时、效率较高的特点，适合处理需要快速响应的外部事件。

在实验过程中，需要将按键对应的 GPIO 引脚配置为中断模式，并设置中断触发方式，如上升沿触发或下降沿触发。同时，还需要初始化中断控制器并使能对应中断源。程序运行后，当按键被按下时，中断触发并改变状态变量，主程序检测到状态变化后执行相应操作。在改进实验中，我将按键中断与 RGB LED 控制结合起来，当按键触发后，串口输出提示信息，同时红色 LED 短暂点亮再熄灭，使中断响应效果更加直观。

我认识到中断程序设计需要注意主程序与中断处理程序之间的配合。中断服务程序一般不宜执行过于复杂或耗时的操作，而是可以通过修改标志变量的方式通知主程序进行后续处理。这样既能保证中断响应速度，也能使程序结构更加稳定。实验还让我理解了 volatile 变量的重要性，因为中断和主程序可能同时访问同一个状态变量，如果没有正确声明，可能会影响程序判断结果。通过本实验，我对嵌入式系统中事件驱动的处理方式有了更深入的认识。

按键轮询实验

1、实验目的

- (1) 熟悉 Eclipse 工具的使用
- (2) 能够导入已有工程
- (3) 能够掌握裸机实验的基本操作步骤
- (4) 通过本实验了解如何使用 C 语言编写裸机程序
- (5) 了解如何通过轮询的方式得到按键状态

2、实验环境

FS6818 开发平台

Eclipse 集成开发环境

虚拟机 Linux 操作系统

3、实验原理

外部设备与中央处理器交互一般有两种手段：轮询和中断。

(1) 轮询 (Polling)

很多 I/O 设备都有一个状态寄存器，用于描述设备当前的工作状态，每当设备状态发生改变时，设备将修改相应状态寄存器位。通过不断查询设备的状态寄存器，CPU 就可以了解设备的状态，从而进行必要的 I/O 操作。为了节约 CPU 资源，查询工作往往不是连续的，而是定时进行。

轮询方式具有简单、易实现、易控制等优势，在很多小型系统中有大量应用。对那些实时敏感性不高、具有大量 CPU 资源的系统来说，轮询方式有很广泛的应用。最典型的用途就是在那些任务比较单一的单片机上，嵌入式系统中也有应用。

轮询的一种典型的实现可能是这样的：`while(TRUE){/…/ select(„timeout);`
`/…/};`当然这里的 `select()` 也可以使用 `poll()` 替换。

轮询方式主要存在以下不足：

<1>增加系统开销。无论是任务轮询还是定时器轮询都需要消耗对应的系统资源。

<2>无法及时感知设备状态变化。在轮询间隔内的设备状态变化只有在下次轮询时才能被发现，这将无法满足对实时性敏感的应用场合。

<3>浪费 CPU 资源。无论设备是否发生状态改变，轮询总在进行。在实际情况中，大多数设备的状态改变通常不会那么频繁，轮询空转将白白浪费 CPU 时间片。

(2) 中断 (Interrupt)

中断，顾名思义，就是打断正在进行中的工作。中断不需要处理器轮询设备的状态，设备在自己发生状态改变时将主动发送一个信号给处理器 (PIC)，后者在接收到这一通知信号时，会挂起当前正在执行的任务转而去处理响应外设的中断请求。中断通知机制通过硬件信号异步唤起处理器的注意，解决了外部设备与处理器之间速度不匹配导致的资源浪费问题。

原理图通按键中断实验相同，此处不再赘述，编程人员通过配置两按键为 GPIO 输入模式即可。

4、实验步骤

这里的软件工具使用 putty 或者 UartAssist 均可。

拷贝路径为 E:\嵌入式系统 FS6818 资料\新 7 寸屏华清远见-嵌入式 ARM 实验箱资料-I-FS6818-JTAG-gt911\程序源码\ARM 裸机实验源码\fs_loop_key 压缩文件到 D 盘 share 文件夹下，再从虚拟机下的文件系统图标 **Files** 中找到 Devices 目录下 Computer 目录下的 mnt 文件夹，如图 2.27，图 2.28 所示，再点击 mnt

文件夹进入下一级目录，直到进入 share 文件夹下，如图 2.29 所示。在 share 文件夹下找到 fs_loop_key 压缩文件，并拷贝此文件到虚拟机中 eclipse 工作目录下。如：/home/linux/workspace/ 目录。

拷贝好文件后，对文件再右键点击 Extract Here 对压缩文件进行解压，如图 2.30 所示。

(1) 打开 Eclipse 开发工具，在 Project Explorer 中导入 fs_loop_key 工程

(注意：工程要放在英文路径下，不能有中文路径)

导入操作可以参照 2.2.1 汇编点灯实验中的第 5 点实验步骤中的导入已有工程操作，如图 2.31~2.33 所示。

(2) 编译工程

编译工程可以参照 2.1.2 章节操作

(3) 调试工程

调试工程可以参照 2.1.3 章节操作

5、实验代码

```
#include <main.h>

#define GPIOA28ALTFN1 (*(volatile unsigned int*)0xC001A024)
#define GPIOA28OUTENB (*(volatile unsigned int*)0xC001A004)
#define GPIOA28OUT      (*(volatile unsigned int*)0xC001A000)

#define GPIOE13ALTFN0 (*(volatile unsigned int*)0xC001E020)
#define GPIOE13OUTENB (*(volatile unsigned int*)0xC001E004)
#define GPIOE13OUT      (*(volatile unsigned int*)0xC001E000)

#define GPIOB12ALTFN0 (*(volatile unsigned int*)0xC001B020)
#define GPIOB12OUTENB (*(volatile unsigned int*)0xC001B004)
#define GPIOB12OUT      (*(volatile unsigned int*)0xC001B000)

void rgb_init(void)
{
```

```
unsigned int reg = 0;
```

```
reg = GPIOA28ALTFN1;
```

```
reg &= ~(0x3 << 28);
```

```
GPIOA28ALTFN1 = reg;
```

```
reg = GPIOA28OUTENB;
```

```
reg &= ~(0x1 << 28);
```

```
reg |= (0x1 << 28);
```

```
GPIOA28OUTENB = reg;
```

```
reg = GPIOE13ALTFN0;
```

```
reg &= ~(0x3 << 26);
```

```
GPIOE13ALTFN0 = reg;
```

```
reg = GPIOE13OUTENB;
```

```
reg &= ~(0x1 << 13);
```

```
reg |= (0x1 << 13);
```

```
GPIOE13OUTENB = reg;
```

```
reg = GPIOB12ALTFN0;
```

```
reg &= ~(0x3 << 24);
```

```
reg |= (0x2 << 24);
```

```
GPIOB12ALTFN0 = reg;
```

```
reg = GPIOB12OUTENB;
```

```
reg &= ~(0x1 << 12);
```

```
reg |= (0x1 << 12);
```

```
GPIOB12OUTENB = reg;
```

```
//初始状态

GPIOA28OUT &= ~(0x1 << 28);

GPIOE13OUT &= ~(0x1 << 13);

GPIOB12OUT &= ~(0x1 << 12);

}

void delay(volatile int count)
{
    while (count--);
}

void set_led(volatile unsigned int* gpio_out, int pin, int state)
{
    if (state)
    {
        *gpio_out |= (0x1 << pin);
    }
    else
    {
        *gpio_out &= ~(0x1 << pin);
    }
}

void r_led(int state)
{
    set_led(&GPIOA28OUT, 28, state);
}

void g_led(int state)
{

```

```
    set_led(&GPIOE13OUT, 13, state);
}

void b_led(int state)
{
    set_led(&GPIOB12OUT, 12, state);
}

void all_led_off(void)
{
    r_led(0);
    g_led(0);
    b_led(0);
}

static void do_system_initial(void)
{
    malloc_init();

    s5p6818_serial_initial();

    led_initial();
    key_initial();
}

int main(int argc, char * argv[])
{
    u32_t keyup = 0;
    u32_t keydown = 0;

    int led_mode = 0;
```

```
do_system_initial();

rgb_init();

led_set_ledb_status(LED_STATUS_OFF);

all_led_off();

s5p6818_serial_write_string(0, "s5p6818 loop key test ...\r\n");

s5p6818_serial_write_string(0, "Press key: RED -> GREEN -> BLUE -> RED\r\n");

while(1)
{
    keyup = 0;

    keydown = 0;

    if(get_key_event(&keyup, &keydown))
    {

        if(keydown != 0)
        {
            if(keydown & KEY_NAME_DOWN)
            {
                all_led_off();

                if(led_mode == 0)
                {
                    r_led(1);

                    s5p6818_serial_write_string(0, "Red LED ON\r\n");
                }

                else if(led_mode == 1)
```

```
{  
  
    g_led(1);  
  
    s5p6818_serial_write_string(0, "Green LED ON\r\n");  
  
}  
  
else if(led_mode == 2)  
  
    {  
  
        b_led(1);  
  
        s5p6818_serial_write_string(0, "Blue LED ON\r\n");  
  
    }  
  
}  
  
}  
  
  
  
  
if(keyup != 0)  
  
    {  
  
        if(keyup & KEY_NAME_DOWN)  
  
            {  
  
                all_led_off();  
  
                s5p6818_serial_write_string(0, "LED OFF\r\n");  
  
  
                led_mode++;  
  
  
                if(led_mode > 2)  
  
                    {  
  
                        led_mode = 0;  
  
                    }  
  
            }  
  
    }  
  
}
```

```
        delay(1000);  
    }  
  
    return 0;  
}
```

6、实验现象

原实验

点击绿色箭头运行测试，可观察到 LED 灯被点亮，如图 2.42 所示。



图 2.42 led 灯被点亮

改进实验

程序下载并运行后，系统首先完成串口、LED 和按键的初始化。初始化完成后，串口调试助手中会显示

s5p6818 loop key test ...

Press key: RED -> GREEN -> BLUE -> RED

这说明程序已经进入按键轮询实验状态。此时 RGB LED 的初始状态全部熄灭。

随后程序进入 while(1) 无限循环，不断调用 get_key_event(&keyup, &keydown) 函数检测按键状态。当没有按键动作时，程序一直处于轮询等待状

态，RGB LED 保持熄灭，串口也不会继续输出新的按键信息。

当第一次按下按键时，程序检测到 keydown 按下事件，根据当前 led_mode 的值点亮红色 LED，同时串口输出：Red LED ON

此时可以观察到红灯点亮，绿灯和蓝灯保持熄灭。当松开按键时，程序检测到 keyup 松开事件，关闭所有 LED，红灯熄灭，同时串口输出：LED OFF

松开按键后，程序将 led_mode 切换到下一种颜色。

再次按下按键时，程序点亮绿色 LED，串口输出：Green LED ON

此时绿灯亮，红灯和蓝灯熄灭。松开按键后，绿灯熄灭，串口再次输出：LED OFF

第三次按下按键时，程序点亮蓝色 LED，串口输出：Blue LED ON

此时蓝灯亮，红灯和绿灯熄灭。松开按键后，蓝灯熄灭，串口输出：LED OFF

之后如果继续按下按键，LED 会重新从红色开始，按照 红灯 → 绿灯 → 蓝灯 → 红灯 的顺序循环变化。

因此，实验中可以观察到：每按下一次按键，当前对应颜色的 LED 点亮；松开按键后，该 LED 熄灭；下一次按下按键时切换到下一种颜色。整个过程中，每次只有一个颜色的 LED 被点亮，不会出现多个颜色同时亮起的情况。



7、心得体会

通过按键轮询实验，我理解了轮询方式检测外部输入的基本思想。轮询方式是指程序在主循环中不断读取按键状态，根据按键按下或松开的情况执行对应操作。这种方式实现简单、逻辑清楚，不需要配置复杂的中断控制器，因此在一些功能较简单、实时性要求不高的嵌入式系统中应用比较广泛。

在原实验中，程序通过轮询按键状态控制 LED 的亮灭，用于验证按键输入功能是否正常。在改进实验中，我增加了 `led_mode` 状态变量，用来记录当前应该点亮的 LED 颜色。每次按下按键时，程序根据 `led_mode` 的值点亮红灯、绿灯或蓝灯；当松开按键时，LED 熄灭，并将 `led_mode` 切换到下一种颜色。这样就实现了按键控制 RGB LED 按红、绿、蓝顺序循环变化的效果。同时，程序还通过串口输出当前 LED 状态，使实验现象更加清晰。

通过本实验，我体会到轮询方式虽然简单，但也存在一定不足。由于程序需要不断循环检测按键状态，即使没有按键动作，CPU 也会一直执行检测代码，这会造成一定资源浪费。同时，如果轮询间隔过长，可能会导致按键响应不及时；如果轮询过于频繁，又会占用较多 CPU 时间。因此，在实际开发中，需要根据系统需求选择合适的按键处理方式。通过与按键中断实验对比，我更加清楚地理解了轮询和中断两种方式的区别：轮询适合简单场景，中断适合实时性要求更高的场景。

A/D 实验

1、实验环境

FS6818 开发平台
Eclipse 集成开发环境
虚拟机 Linux 操作系统

2、实验目的

掌握模数转换 ADC 的原理；
掌握 S5P6818 处理器的 A/D 转换功能；

3、实验原理

如图 2.43 所示，ADC 电路连接如图所示，利用一个电位计输出电压到 S5P6818 的 ADC0 引脚。输入的电压范围是 0~1.8V。旋转电位器 XR 使 ADC0 和 GND 两端的电压发生变化，即 ADC0 引脚采集变化的模拟电压，即 ADC 控制器的数据寄存器可输出对应二进制数值。

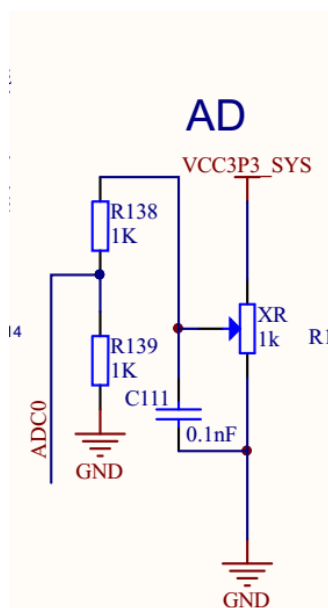


图 2.43 分压电路

4、实验内容

寄存器设置：
上电使能 ADC 控制器
选择转换通道。

使能 ADC 读操作和设置分频因子
设置工作模式和转换分辨率

编写软件程序，实现电压值的获取、显示。程序主要是对 S5P6818 中的 A/D 模块进行操作，所以软件程序也主要是对 A/D 模块中的寄存器进行操作，其中包括对 ADC 控制寄存器（ADCCON）、ADC 数据寄存器（ADC DAT）的读/写操作。同时为了观察转换结果，可以通过串口在超级终端里面观察。

5、实验步骤

这里的软件工具使用 putty 或者 UartAssist 均可。

拷贝路径为 E:\嵌入式系统 FS6818 资料\新 7 寸屏华清远见-嵌入式 ARM 实验箱资料-I-FS6818-JTAG-gt911\程序源码\ARM 裸机实验源码\fs_adc 压缩文件到 D 盘 share 文件夹下，再从虚拟机下的文件图标中找到 Devices 目录下 Computer 目录下的 mnt 文件夹，如图 2.27，图 2.28 所示，再点击 mnt 文件夹进入下一级目录，直到进入 share 文件夹下，如图 2.29 所示。在 share 文件夹下找到 fs_adc 压缩文件，并拷贝此文件到虚拟机中 eclipse 工作目录下。如 /home/linux/workspace/目录。

拷贝好文件后，对文件再右键点击 Extract Here 对压缩文件进行解压，如图 2.30 所示。

(1) 打开 Eclipse 开发工具，在 Project Explorer 中导入 fs_serial 工程

(注意：工程要放在英文路径下，不能有中文路径)

导入操作可以参照 2.2.1 汇编点灯实验中的第 5 点实验步骤中的导入已有工程操作，如图 2.31~2.33 所示。

(2) 编译工程

编译工程可以参照 2.1.2 章节操作

(3) 调试工程

调试工程可以参照 2.1.3 章节操作

6、实验代码

```
#include <main.h>

#define ADCCON                0xC0053000

#define ADCDAT                0xC0053004

#define ADCINTENB             0xC0053008

#define ADCINTCLR             0xC005300C

#define PRESCALERCON          0xC0053010

#define rADCCON                (*(volatile unsigned int *)ADCCON)

#define rADCDAT                (*(volatile unsigned int *)ADCDAT)

#define rADCINTENB             (*(volatile unsigned int *)ADCINTENB)

#define rADCINTCLR             (*(volatile unsigned int *)ADCINTCLR)

#define rPRESCALERCON          (*(volatile unsigned int *)PRESCALERCON)

#define RESET_ID_ADC          60

#define APEN_BITP              (15)

#define PRES_BITP              (0)

/* ADCCON */

#define DATA_SEL_VAL          (0)      /* 0:5clk, 1:4clk, 2:3clk, 3:2clk, 4:1clk, 5:0clk */

#define CLK_CNT_VAL            (6)      /* 28nm ADC */

#define DATA_SEL_BITP          (10)     /* 13:10 */

#define CLK_CNT_BITP            (6)      /* 9:6 */

#define ASEL_BITP               (3)

#define ADCON_STBY              (2)

#define ADEN_BITP               (0)
```

```

/* ADCINTENB */

#define AIEN_BITP      (0)

/* ADCINTCLR */

#define AICL_BITP      (0)


struct NX_RSTCON_RegisterSet
{
    volatile unsigned int    REGRST[(69+31)>>5];
};

static struct NX_RSTCON_RegisterSet *__g_pRegister;

typedef enum
{
    RSTCON_ASSERT    = 0UL,
    RSTCON_NEGATE     = 1UL
}RSTCON;

void NX_RSTCON_SetRST(unsigned int RSTIndex, RSTCON STATUS)
{
    unsigned int regvalue;

    regvalue    = *(volatile unsigned int *)(&__g_pRegister->REGRST[RSTIndex >> 5]);

    regvalue    &= ~(1UL << (RSTIndex & 0x1f));
    regvalue    |= (STATUS & 0x01) << (RSTIndex & 0x1f);

    *(volatile unsigned int *)(&__g_pRegister->REGRST[RSTIndex >> 5]) = (unsigned int )(regvalue);
}

```

```
char string[1024] = "";

char* itoa(unsigned int value)
{

    int i = 0, j = 0;

    int len = 0;

    do
    {
        if(value == 0)
        {
            string[i++] = 0;
        }
        else
        {
            char c = value % 10 + '0';

            string[i++] = c;

            value /= 10;
        }
    }

    while(value != 0);

    string[i] = '\0';

    len = strlen(string);

    for(j=0; j<len/2; j++)
    {
        char temp = string[j];

        string[j] = string[len-j-1];

        string[len-j-1] = temp;
    }

    return string;
}
```

```
}

static void mydelay_ms(int time)
{
    int i, j;
    while(time--)
    {
        for (i = 0; i < 5; i++)
            for (j = 0; j < 514; j++);
    }
}

void adc_init(char ch)
{
    unsigned int adccon = 0;
    unsigned int reg = 0;

    s5p6818_serial_write_string(0, "\n adc init!!\n\r");

    unsigned int RSTIndex = RESET_ID_ADC;
    __g_pRegister = (struct NX_RSTCON_RegisterSet *)((void *)0xC0012000);
    NX_RSTCON_SetRST(RSTIndex, RSTCON_ASSERT);
    NX_RSTCON_SetRST(RSTIndex, RSTCON_NEGATE);

    adccon = ((DATA_SEL_VAL & 0xf) << DATA_SEL_BITP) |
              ((CLK_CNT_VAL & 0xf) << CLK_CNT_BITP) |
              (0 << ADCON_STBY);

    rADCCON = adccon;

    reg = rPRESCALERCON;
```

```
reg &= ~(0x3FF << PRES_BITP);

reg |= 0xFF << PRES_BITP;

reg |= 0x1 << APEN_BITP;

rPRESCALERCON = reg;


rADCINTCLR = (1 << AICL_BITP);

rADCINTENB = (1 << AIEN_BITP);


reg = rADCCON;

reg &= ~(0x7 << ASEL_BITP);

reg |= (ch << ASEL_BITP);

rADCCON = reg;
}

int tester_adc(int argc, char * argv[])
{

    unsigned int reg = 0;

    unsigned int data = 0;


    unsigned int data1 = 0;

    unsigned int data10 = 0;

    unsigned int data100 = 0;


    s5p6818_serial_write_string(0, "\n adc test!!\n\r");


    while (1) {

        reg = rADCCON;

        reg |= (1 << ADEN_BITP);

        rADCCON = reg;
```

```
data = (rADC_DAT & 0xffff);

// s5p6818_serial_write_string(0, "data= ");
// s5p6818_serial_write_string(0, itoa(data));
// s5p6818_serial_write_string(0, "\n");

data100 = data / 4;

data10 = data100 / 10;

data1 = data100 % 10;

// s5p6818_serial_write_string(0, "F2");
// s5p6818_serial_write_string(0, itoa(data));

s5p6818_serial_write_string(0, "温度为");
s5p6818_serial_write_string(0, itoa(data10));
s5p6818_serial_write_string(0, ".");
s5p6818_serial_write_string(0, itoa(data1));
s5p6818_serial_write_string(0, "°C");
s5p6818_serial_write_string(0, "\n");

s5p6818_serial_write_string(0, "压强为");
s5p6818_serial_write_string(0, itoa(data100));
s5p6818_serial_write_string(0, "Pa");
s5p6818_serial_write_string(0, "\n");

mydelay_ms(2000);

}
```

```

return 0;
}

```

7、实验现象

原实验

点击运行按钮



通过串口终端观察打印数据信息，旋转 A53 开发板左上的电位器，如下图 2.44 中红色圆圈位置。观察其电压的变化，如图 2.45 所示。



图 2.44 开发板上电位器位置

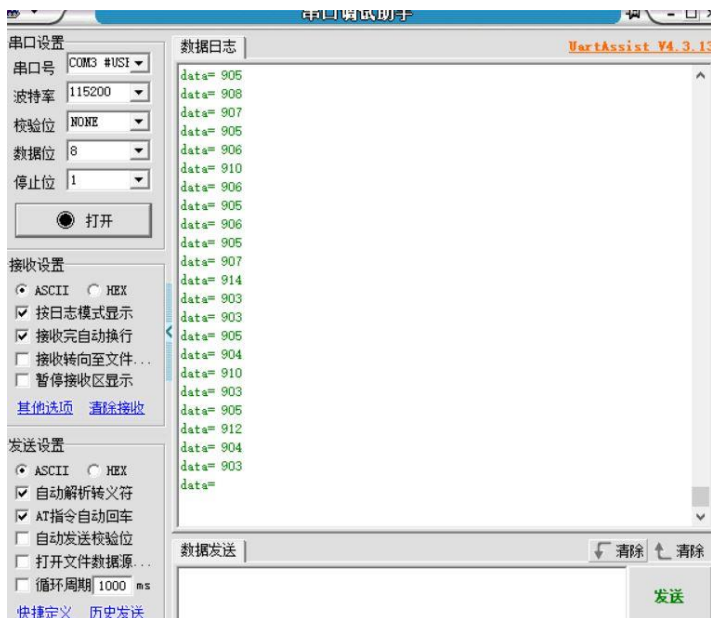


图 2.45 打印 ADC 转换的数据

改进实验

程序运行后，ADC 模块开始采集电位器输出的模拟电压值，并将采集到的 ADC 数据进行简单换算后，通过串口输出为“温度”和“压强”数据。串口终端中可以连续看到类似如下信息：

压强为 218Pa 温度为 21.8℃

压强为 219Pa 温度为 21.9℃

调节电位器前，串口显示的数据大多稳定在：

压强约为 218Pa~219Pa

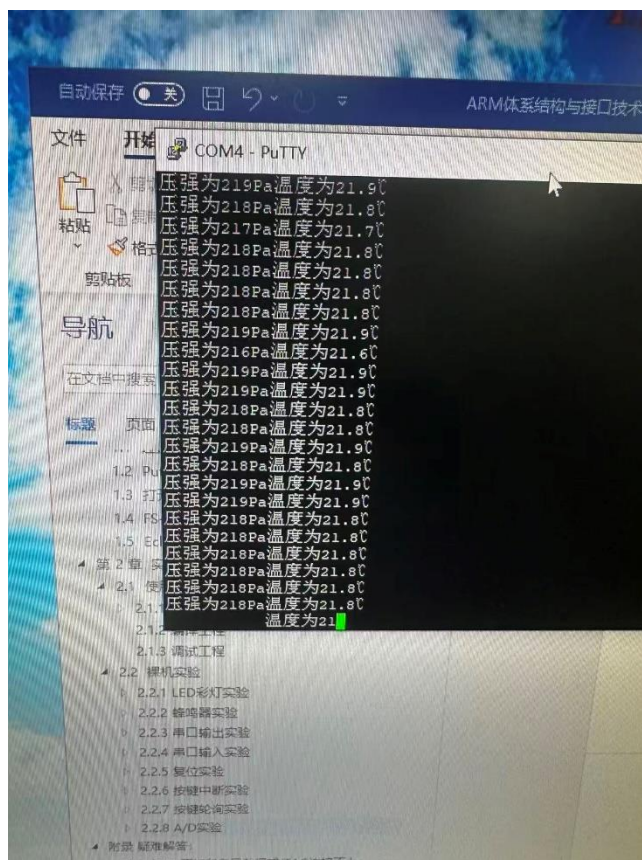
温度约为 21.8℃~21.9℃

调节电位器后，ADC 输入电压发生变化，串口显示的数据随之增大，变为：

压强约为 223Pa~225Pa

温度约为 22.3℃~22.5℃

由实验结果可以看出，调节电位器后，串口输出的数值发生了明显变化，说明 ADC 能够采集电位器输出的模拟电压，并将模拟量转换为数字量后通过串口显示出来。电位器调节前后，显示的温度和压强数值随 ADC 采样值变化而变化，验证了 ADC 采样功能正常。



通过 A/D 实验，我学习了 ADC 模块将模拟信号转换为数字信号的基本过程。嵌入式系统在实际应用中经常需要采集外部模拟量，例如电压、温度、光照强度、压力等，而 CPU 本身只能处理数字信号，因此需要通过 ADC 将模拟电压转换为对应的数字量。本实验中，电位器输出的模拟电压接入 S5P6818 的 ADC 输入引脚，程序通过配置 ADC 相关寄存器完成采样和转换，并将转换结果通过串口输出。

程序启动 ADC 转换后，从数据寄存器中读取转换结果，再根据需要进行显示或换算。通过旋转电位器，可以改变 ADC 输入端的模拟电压，从而观察到串口输出的数据随之变化。这个现象说明 ADC 模块能够正确采集外部模拟电压变化，并将其转换为数字数据。

79